The content credentials are only supported in video files generated by batch synthesis of text to speech avatar, and only `mp4` file format is supported.

Use content credentials in your solution

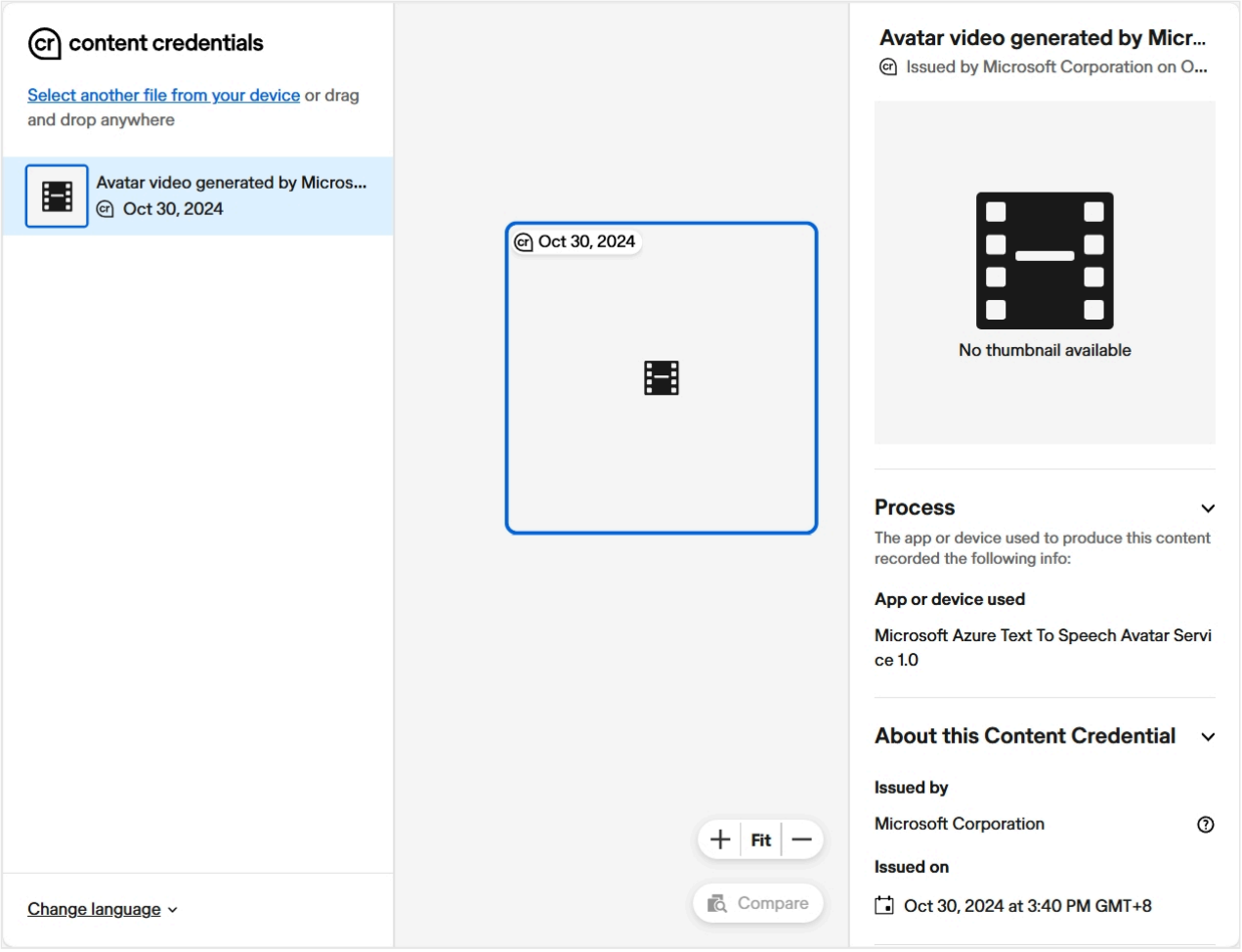
You can use content credentials by ensuring that your Azure text to speech avatar generated video files have content credentials.

No extra setup is necessary. Content credentials are automatically applied to all applicable videos generated by Azure text to speech avatar.

Check that a video file has content credentials

You can check the content credentials of a text to speech avatar using either of the following methods:

- **Content credentials verify webpage** (contentcredentials.org/verify): This tool lets users inspect the content credentials of content like a video. If an uploaded video of a text to speech avatar is created with the Speech service, the tool will show that and display the issued date and time of the content credentials. This screenshot shows an example avatar video upload result where the content credentials were issued by Microsoft.



This page shows that a video generated by Azure text to speech avatar has content credentials issued by Microsoft.

- **Content Authenticity Initiative (CAI) open-source tools**: The CAI provides multiple open-source tools that can be used to validate and display C2PA content credentials. Find the right tool for your application and get started here.

Next steps

- Use batch synthesis for text to speech avatar

How to use text to speech avatar with real-time synthesis

This guide shows you how to use text to speech avatar with real-time synthesis. The avatar video is generated almost instantly after you enter text.

Prerequisites

You need:

- **Azure subscription:** Create one for free.
- **Speech resource:** Create a speech resource in the Azure portal. Select the **Standard S0** pricing tier to access avatars.
- **Speech resource key and region:** After deployment, select **Go to resource** to view and manage your keys.

Set up environment

To use real-time avatar synthesis, install the Speech SDK for JavaScript for your webpage. See Install the Speech SDK.

Real-time avatar works on these platforms and browsers:

Expand table

Platform	Chrome	Microsoft Edge	Safari	Firefox	Opera
Windows	Y	Y	N/A	Y[1]	Y
Android	Y	Y	N/A	Y[1][2]	N
iOS	Y	Y	Y	Y	Y
macOS	Y	Y	Y	Y[1]	Y

[1] Doesn't work with ICE server by Communication Service, but works with Coturn. [2] Background transparency doesn't work.

Select text to speech language and voice

Speech service supports many languages and voices. See the full list or try them in the Voice Gallery.

To match your input text and use a specific voice, set the `SpeechSynthesisLanguage` or `SpeechSynthesisVoiceName` properties in the `SpeechConfig` object:

JavaScript

```
const speechConfig = SpeechSDK.SpeechConfig.fromSubscription("YourSpeechKey",
"YourSpeechRegion");
// Set either the `SpeechSynthesisVoiceName` or `SpeechSynthesisLanguage`.
speechConfig.speechSynthesisLanguage = "en-US";
speechConfig.speechSynthesisVoiceName = "en-US-AvaMultilingualNeural";
```

All neural voices are multilingual and fluent in their own language and English. For example, if you select **es-ES-ElviraNeural** and enter English text, the avatar speaks English with a Spanish accent.

If the voice doesn't support the input language, the Speech service doesn't create audio. See Language and voice support for the full list.

Default voice selection:

- If you don't set `SpeechSynthesisVoiceName` or `SpeechSynthesisLanguage`, the default voice in `en-US` is used.
- If you set only `SpeechSynthesisLanguage`, the default voice in that locale is used.
- If you set both, `SpeechSynthesisVoiceName` takes priority.
- If you use SSML to set the voice, both properties are ignored.

Select avatar character and style

See supported avatar characters and styles.

Set avatar character and style:

JavaScript

```
const avatarConfig = new SpeechSDK.AvatarConfig(
    "lisa", // Set avatar character here.
    "casual-sitting", // Set avatar style here.
);
```

Set up connection to real-time avatar

Real-time avatar uses the WebRTC protocol to stream video. Set up the connection with the avatar service using a WebRTC peer connection.

First, create a WebRTC peer connection object. WebRTC is peer-to-peer and relies on an ICE server for network relay. Speech service provides a REST API to get ICE server info. We recommend fetching ICE server details from Speech service, but you can use your own.

Sample request to fetch ICE info:

HTTP

```
GET /cognitiveservices/avatar/relay/token/v1 HTTP/1.1

Host: westus2.tts.speech.microsoft.com
Ocp-Apim-Subscription-Key: YOUR_RESOURCE_KEY
```

Create the WebRTC peer connection using the ICE server URL, username, and credential from the previous response:

JavaScript

```
// Create WebRTC peer connection
peerConnection = new RTCPeerConnection({
    iceServers: [{
        urls: [ "Your ICE server URL" ],
        username: "Your ICE server username",
        credential: "Your ICE server credential"
    }]
})
```

> **Note**
>
> The ICE server URL can start with `turn` (for example, `turn:relay.communication.microsoft.com:3478`) or `stun` (for example, `stun:relay.communication.microsoft.com:3478`). For `urls`, include only the `turn` URL.

Next, set up the video and audio player elements in the `ontrack` callback of the peer connection. This callback runs twice—once for video, once for audio. Create both player elements in the callback:

JavaScript

```
// Fetch WebRTC video/audio streams and mount them to HTML video/audio player
elements
peerConnection.ontrack = function (event) {
    if (event.track.kind === 'video') {
        const videoElement = document.createElement(event.track.kind)
        videoElement.id = 'videoPlayer'
        videoElement.srcObject = event.streams[0]
```

```
        videoElement.autoplay = true
    }

    if (event.track.kind === 'audio') {
        const audioElement = document.createElement(event.track.kind)
        audioElement.id = 'audioPlayer'
        audioElement.srcObject = event.streams[0]
        audioElement.autoplay = true
    }
}

// Offer to receive one video track, and one audio track
peerConnection.addTransceiver('video', { direction: 'sendrecv' })
peerConnection.addTransceiver('audio', { direction: 'sendrecv' })
```

Then, use Speech SDK to create an avatar synthesizer and connect to the avatar service with the peer connection:

JavaScript

```
// Create avatar synthesizer
var avatarSynthesizer = new SpeechSDK.AvatarSynthesizer(speechConfig, avatarConfig)

// Start avatar and establish WebRTC connection
avatarSynthesizer.startAvatarAsync(peerConnection).then(
    (r) => { console.log("Avatar started.") }
).catch(
    (error) => { console.log("Avatar failed to start. Error: " + error) }
);
```

The real-time API disconnects after 5 minutes of idle or after 30 minutes of connection. To keep the avatar running longer, enable automatic reconnect. See this JavaScript sample code (search "auto reconnect").

Synthesize talking avatar video from text input

After setup, the avatar video plays in your browser. The avatar blinks and moves slightly, but doesn't speak until you send text input.

Send text to the avatar synthesizer to make the avatar speak:

JavaScript

```
var spokenText = "I'm excited to try text to speech avatar."
avatarSynthesizer.speakTextAsync(spokenText).then(
    (result) => {
        if (result.reason === SpeechSDK.ResultReason.SynthesizingAudioCompleted) {
            console.log("Speech and avatar synthesized to video stream.")
        } else {
```

```
            console.log("Unable to speak. Result ID: " + result.resultId)
            if (result.reason === SpeechSDK.ResultReason.Canceled) {
                let cancellationDetails =
SpeechSDK.CancellationDetails.fromResult(result)
                console.log(cancellationDetails.reason)
                if (cancellationDetails.reason ===
SpeechSDK.CancellationReason.Error) {
                    console.log(cancellationDetails.errorDetails)
                }
            }
        }
}).catch((error) => {
    console.log(error)
    avatarSynthesizer.close()
});
```

Close the real-time avatar connection

To avoid extra costs, close the connection when you're done:

- Closing the browser releases the WebRTC peer connection and closes the avatar connection after a few seconds.
- The connection closes automatically if the avatar is idle for 5 minutes.
- You can close the avatar connection manually:

JavaScript

```
avatarSynthesizer.close()
```

Edit background

Set background color

Set the background color of the avatar video using the `backgroundColor` property of `AvatarConfig`:

JavaScript

```
const avatarConfig = new SpeechSDK.AvatarConfig(
    "lisa", // Set avatar character here.
    "casual-sitting", // Set avatar style here.
)
avatarConfig.backgroundColor = '#00FF00FF' // Set background color to green
```

> **Note**
>
> The color string should be in the format `#RRGGBBAA`. The alpha channel (`AA`) is ignored—transparent backgrounds aren't supported for real-time avatar.

Set background image

Set the background image using the `backgroundImage` property of `AvatarConfig`. Upload your image to a public URL and assign it to `backgroundImage`:

JavaScript

```
const avatarConfig = new SpeechSDK.AvatarConfig(
    "lisa", // Set avatar character here.
    "casual-sitting", // Set avatar style here.
)
avatarConfig.backgroundImage = "https://www.example.com/1920-1080-image.jpg" // A public accessiable URL of the image.
```

Set background video

The API doesn't support background video directly, but you can customize the background on the client side:

- Set the background color to green (for easy matting).
- Create a canvas element the same size as the avatar video.
- For each frame, set green pixels to transparent and draw the frame to the canvas.
- Hide the original video.

This gives you a transparent avatar on a canvas. See JavaScript sample code.

You can then place any dynamic content (like a video) behind the canvas.

Crop video

The avatar video is 16:9 by default. To crop to a different aspect ratio, specify the rectangle area using the coordinates of the top-left and bottom-right corners:

JavaScript

```
const videoFormat = new SpeechSDK.AvatarVideoFormat()
const topLeftCoordinate = new SpeechSDK.Coordinate(640, 0) // coordinate of top-left vertex, with X=640, Y=0
```

```
const bottomRightCoordinate = new SpeechSDK.Coordinate(1320, 1080) // coordinate of bottom-right vertex, with X=1320, Y=1080
videoFormat.setCropRange(topLeftCoordinate, bottomRightCoordinate)
const avatarConfig = new SpeechSDK.AvatarConfig(
    "lisa", // Set avatar character here.
    "casual-sitting", // Set avatar style here.
    videoFormat, // Set video format here.
)
```

For a full sample, see our code example and search for `crop`.

Code samples

Find text to speech avatar code samples in the Speech SDK GitHub repository. These samples show how to use real-time avatars in web and mobile apps:

- **Server + client**
  - Python (server) + JavaScript (client)
  - C# (server) + JavaScript (client)
- **Client only**
  - JavaScript
  - Android
  - iOS

Next steps

- What is text to speech avatar
- Install the Speech SDK

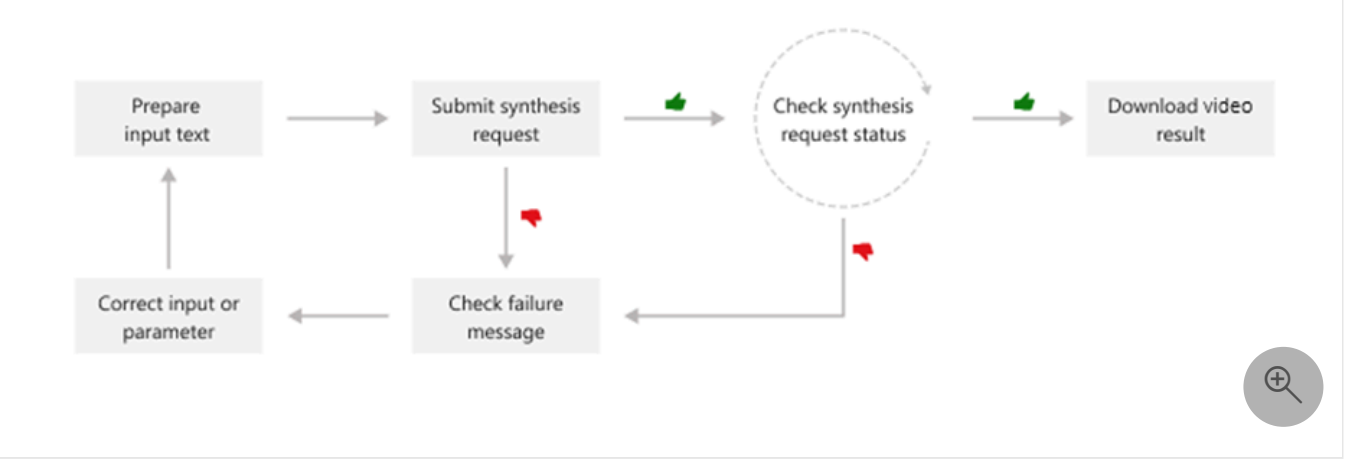
Last updated on 11/09/2025

Use batch synthesis for text to speech avatar

The batch synthesis API for text to speech avatar lets you synthesize text asynchronously into a talking avatar as a video file. Publishers and video content platforms can use this API to create avatar video content in a batch. That approach can be suitable for different use cases like training materials, presentations, or advertisements.

The synthetic avatar video will be generated asynchronously after the system receives text input. The generated video output can be downloaded in batch mode synthesis. You submit text for synthesis, poll for the synthesis status, and download the video output when the status shows success. The text input formats must be plain text or Speech Synthesis Markup Language (SSML) text.

This diagram provides a high-level overview of the workflow.



To run batch synthesis, you can use the following REST API operations.

Expand table

Operation	Method	REST API call
Create batch synthesis	PUT	avatar/batchsyntheses/{SynthesisId}?api-version=2024-08-01
Get batch synthesis	GET	avatar/batchsyntheses/{SynthesisId}?api-version=2024-08-01
List batch synthesis	GET	avatar/batchsyntheses/?api-version=2024-08-01
Delete batch synthesis	DELETE	avatar/batchsyntheses/{SynthesisId}?api-version=2024-08-01

You can refer to the code samples on GitHub.

Create a batch synthesis request

Some properties in JSON format are required when you create a new batch synthesis job. Other properties are optional. The batch synthesis response includes other properties to provide information about the synthesis status and results. For example, the `outputs.result` property has the location from where you can download a video file containing the avatar video. From `outputs.summary`, you can get the summary and debug details.

To submit a batch synthesis request, construct the HTTP POST request body following these instructions:

- Set the required `inputKind` property.
- If the `inputKind` property is set to `PlainText`, you must also set the `voice` property in the `synthesisConfig`. In the following example, the `inputKind` is set to `SSML`, so the `speechSynthesis` isn't set.
- Set the required `SynthesisId` property. Choose a unique `SynthesisId` for the same speech resource. The `SynthesisId` can be a string of 3 to 64 characters, including letters, numbers, '-', or '_', with the condition that it must start and end with a letter or number.
- Set the required `talkingAvatarCharacter` and `talkingAvatarStyle` properties. You can find supported avatar characters and styles here.
- Optionally, you can set the `videoFormat`, `backgroundColor`, and other properties. For more information, see batch synthesis properties.

> **Note**
>
> The maximum JSON payload size accepted is 500 kilobytes.
>
> Each Speech resource can have up to 200 batch synthesis jobs running concurrently.
>
> The maximum length for the output video is currently 20 minutes, with potential increases in the future.

To make an HTTP PUT request, use the URI format shown in the following example. Replace `YourSpeechKey` with your Speech resource key, `YourSpeechRegion` with your Speech resource region, and set the request body properties as described previously.

Azure CLI

```
curl -v -X PUT -H "Ocp-Apim-Subscription-Key: YourSpeechKey" -H "Content-Type:
application/json" -d '{
    "inputKind": "SSML",
    "inputs": [
        {
         "content": "<speak version='\''1.0'\'' xml:lang='\''en-US'\''><voice
name='\''en-US-AvaMultilingualNeural'\''>The rainbow has seven colors.</voice>
```

```
</speak>"
        }
    ],
    "avatarConfig": {
        "talkingAvatarCharacter": "lisa",
        "talkingAvatarStyle": "graceful-sitting"
    }
}'  "https://YourSpeechRegion.api.cognitive.microsoft.com/avatar/batchsyntheses/my-job-01?api-version=2024-08-01"
```

You should receive a response body in the following format:

JSON

```
{
    "id": "my-job-01",
    "internalId": "5a25b929-1358-4e81-a036-33000e788c46",
    "status": "NotStarted",
    "createdDateTime": "2024-03-06T07:34:08.9487009Z",
    "lastActionDateTime": "2024-03-06T07:34:08.9487012Z",
    "inputKind": "SSML",
    "customVoices": {},
    "properties": {
        "timeToLiveInHours": 744,
    },
    "avatarConfig": {
        "talkingAvatarCharacter": "lisa",
        "talkingAvatarStyle": "graceful-sitting",
        "videoFormat": "Mp4",
        "videoCodec": "hevc",
        "subtitleType": "soft_embedded",
        "bitrateKbps": 2000,
        "customized": false
    }
}
```

The `status` property should progress from `NotStarted` status to `Running` and finally to `Succeeded` or `Failed`. You can periodically call the GET batch synthesis API until the returned status is `Succeeded` or `Failed`.

Get batch synthesis

To get the status of a batch synthesis job, make an HTTP GET request using the URI as shown in the following example.

Replace `YourSynthesisId` with your batch synthesis ID, `YourSpeechKey` with your Speech resource key, and `YourSpeechRegion` with your Speech resource region.

Azure CLI

```
curl -v -X GET
"https://YourSpeechRegion.api.cognitive.microsoft.com/avatar/batchsyntheses/YourSynt
hesisId?api-version=2024-08-01" -H "Ocp-Apim-Subscription-Key: YourSpeechKey"
```

You should receive a response body in the following format:

JSON

```
{
    "id": "my-job-01",
    "internalId": "5a25b929-1358-4e81-a036-33000e788c46",
    "status": "Succeeded",
    "createdDateTime": "2024-03-06T07:34:08.9487009Z",
    "lastActionDateTime": "2024-03-06T07:34:12.5698769",
    "inputKind": "SSML",
    "customVoices": {},
    "properties": {
        "timeToLiveInHours": 744,
        "sizeInBytes": 344460,
        "durationInMilliseconds": 2520,
        "succeededCount": 1,
        "failedCount": 0,
        "billingDetails": {
            "neuralCharacters": 29,
            "talkingAvatarDurationSeconds": 2
        }
    },
    "avatarConfig": {
        "talkingAvatarCharacter": "lisa",
        "talkingAvatarStyle": "graceful-sitting",
        "videoFormat": "Mp4",
        "videoCodec": "hevc",
        "subtitleType": "soft_embedded",
        "bitrateKbps": 2000,
        "customized": false
    },
    "outputs": {
        "result": "https://stttssvcprodusw2.blob.core.windows.net/batchsynthesis-
output/xxxxx/xxxxx/0001.mp4?SAS_Token",
        "summary": "https://stttssvcprodusw2.blob.core.windows.net/batchsynthesis-
output/xxxxx/xxxxx/summary.json?SAS_Token"
    }
}
```

From the `outputs.result` field, you can download a video file containing the avatar video. The `outputs.summary` field lets you download the summary and debug details. For more information on batch synthesis results, see batch synthesis results.

List batch synthesis

To list all batch synthesis jobs for your Speech resource, make an HTTP GET request using the URI as shown in the following example.

Replace `YourSpeechKey` with your Speech resource key and `YourSpeechRegion` with your Speech resource region. Optionally, you can set the `skip` and `top` (page size) query parameters in the URL. The default value for `skip` is 0, and the default value for `maxpagesize` is 100.

Azure CLI

```
curl -v -X GET
"https://YourSpeechRegion.api.cognitive.microsoft.com/avatar/batchsyntheses?
skip=0&maxpagesize=2&api-version=2024-08-01" -H "Ocp-Apim-Subscription-Key:
YourSpeechKey"
```

You receive a response body in the following format:

JSON


```
{
    "value": [
        {
            "id": "my-job-02",
            "internalId": "14c25fcf-3cb6-4f46-8810-ecad06d956df",
            "status": "Succeeded",
            "createdDateTime": "2024-03-06T07:52:23.9054709Z",
            "lastActionDateTime": "2024-03-06T07:52:29.3416944",
            "inputKind": "SSML",
            "customVoices": {},
            "properties": {
                "timeToLiveInHours": 744,
                "sizeInBytes": 502676,
                "durationInMilliseconds": 2950,
                "succeededCount": 1,
                "failedCount": 0,
                "billingDetails": {
                    "neuralCharacters": 32,
                    "talkingAvatarDurationSeconds": 2
                }
            },
            "avatarConfig": {
                "talkingAvatarCharacter": "lisa",
                "talkingAvatarStyle": "casual-sitting",
                "videoFormat": "Mp4",
                "videoCodec": "h264",
                "subtitleType": "soft_embedded",
                "bitrateKbps": 2000,
                "customized": false
            },
```

```
            "outputs": {
                "result":
"https://stttssvcprodusw2.blob.core.windows.net/batchsynthesis-
output/xxxxx/xxxxx/0001.mp4?SAS_Token",
                "summary":
"https://stttssvcprodusw2.blob.core.windows.net/batchsynthesis-
output/xxxxx/xxxxx/summary.json?SAS_Token"
            }
        },
        {
            "id": "my-job-01",
            "internalId": "5a25b929-1358-4e81-a036-33000e788c46",
            "status": "Succeeded",
            "createdDateTime": "2024-03-06T07:34:08.9487009Z",
            "lastActionDateTime": "2024-03-06T07:34:12.5698769",
            "inputKind": "SSML",
            "customVoices": {},
            "properties": {
                "timeToLiveInHours": 744,
                "sizeInBytes": 344460,
                "durationInMilliseconds": 2520,
                "succeededCount": 1,
                "failedCount": 0,
                "billingDetails": {
                    "neuralCharacters": 29,
                    "talkingAvatarDurationSeconds": 2
                }
            },
            "avatarConfig": {
                "talkingAvatarCharacter": "lisa",
                "talkingAvatarStyle": "graceful-sitting",
                "videoFormat": "Mp4",
                "videoCodec": "hevc",
                "subtitleType": "soft_embedded",
                "bitrateKbps": 2000,
                "customized": false
            },
            "outputs": {
                "result":
"https://stttssvcprodusw2.blob.core.windows.net/batchsynthesis-
output/xxxxx/xxxxx/0001.mp4?SAS_Token",
                "summary":
"https://stttssvcprodusw2.blob.core.windows.net/batchsynthesis-
output/xxxxx/xxxxx/summary.json?SAS_Token"
            }
        }
    ],
    "nextLink":
"https://YourSpeechRegion.api.cognitive.microsoft.com/avatar/batchsyntheses/?api-
version=2024-08-01&skip=2&maxpagesize=2"
}
```


From `outputs.result`, you can download a video file containing the avatar video. From `outputs.summary`, you can get the summary and debug details. For more information, see batch

synthesis results.

The `value` property in the JSON response lists your synthesis requests. The list is paginated, with a maximum page size of 100. The `nextLink` property is provided as needed to get the next page of the paginated list.

Get batch synthesis results file

Once you get a batch synthesis job with `status` of "Succeeded", you can download the video output results. Use the URL from the `outputs.result` property of the get batch synthesis response.

To get the batch synthesis results file, make an HTTP GET request using the URI as shown in the following example. Replace `YourOutputsResultUrl` with the URL from the `outputs.result` property of the get batch synthesis response. Replace `YourSpeechKey` with your Speech resource key.

Azure CLI

```
curl -v -X GET "YourOutputsResultUrl" -H "Ocp-Apim-Subscription-Key: YourSpeechKey"
> output.mp4
```

To get the batch synthesis summary file, make an HTTP GET request using the URI as shown in the following example. Replace `YourOutputsResultUrl` with the URL from the `outputs.summary` property of the get batch synthesis response. Replace `YourSpeechKey` with your Speech resource key.

Azure CLI

```
curl -v -X GET "YourOutputsSummaryUrl" -H "Ocp-Apim-Subscription-Key: YourSpeechKey"
> summary.json
```

The summary file has the synthesis results for each text input. Here's an example summary.json file:

JSON

```
{
  "jobID": "5a25b929-1358-4e81-a036-33000e788c46",
  "status": "Succeeded",
  "results": [
    {
      "texts": [
        "<speak version='1.0' xml:lang='en-US'><voice name='en-US-
AvaMultilingualNeural'>The rainbow has seven colors.</voice></speak>"
```

```
      ],
      "status": "Succeeded",
      "videoFileName": "244a87c294b94ddeb3dbaccee8ffa7eb/5a25b929-1358-4e81-a036-33000e788c46/0001.mp4",
      "TalkingAvatarCharacter": "lisa",
      "TalkingAvatarStyle": "graceful-sitting"
    }
  ]
}
```

Delete batch synthesis

After you get the audio output results and no longer need the batch synthesis job history, you can delete it. The Speech service keeps each synthesis history for up to 31 days or the duration specified by the request's `timeToLiveInHours` property, whichever comes sooner. The date and time of automatic deletion for synthesis jobs with a status of "Succeeded" or "Failed" is calculated as the sum of the `lastActionDateTime` and `timeToLive` properties.

To delete a batch synthesis job, make an HTTP DELETE request using the following URI format. Replace `YourSynthesisId` with your batch synthesis ID, `YourSpeechKey` with your Speech resource key, and `YourSpeechRegion` with your Speech resource region.

Azure CLI

```
curl -v -X DELETE "https://YourSpeechRegion.api.cognitive.microsoft.com/avatar/batchsyntheses/YourSynthesisId?api-version=2024-08-01" -H "Ocp-Apim-Subscription-Key: YourSpeechKey"
```

The response headers include `HTTP/1.1 204 No Content` if the delete request was successful.

Next steps

- Batch synthesis properties
- Use batch synthesis for text to speech avatar
- What is text to speech avatar

Last updated on 11/08/2025

Batch synthesis properties for text to speech avatar

08/07/2025

Batch synthesis properties can be grouped as: avatar related properties, batch job related properties, and text to speech related properties, which are described in the following tables.

Some properties in JSON format are required when you create a new batch synthesis job. Other properties are optional. The batch synthesis response includes other properties to provide information about the synthesis status and results. For example, the `outputs.result` property contains the location from where you can download a video file containing the avatar video. From `outputs.summary`, you can access the summary and debug details.

Avatar properties

The following table describes the avatar properties.

Expand table

Property	Description
avatarConfig.talkingAvatarCharacter	The character name of the talking avatar. For standard avatar, the supported avatar characters can be found here. For custom avatar, specify the avatar model name. This property is required.
avatarConfig.talkingAvatarStyle	The style name of the talking avatar. For standard avatar, the supported avatar styles can be found here. For custom avatar, this property should be omitted. This property is required for standard avatar.
avatarConfig.customized	A bool value indicating whether the avatar to be used is customized avatar or not. True for customized avatar, and false for standard avatar. This property is optional, and the default value is `false`.

Property	Description
avatarConfig.videoFormat	The format for output video file could be mp4 or webm. The `webm` format is required for a transparent background. This property is optional, and the default value is mp4.
avatarConfig.videoCodec	The codec for output video, could be h264, hevc, vp9 or av1. Vp9 is required for a transparent background. The synthesis speed is slower with the vp9 codec because vp9 encoding is slower. This property is optional, and the default value is hevc.
avatarConfig.bitrateKbps	The bitrate for output video, which is integer value, with unit kbps. This property is optional, and the default value is 2000.
avatarConfig.videoCrop	This property lets you crop the video output, which means to output a rectangle subarea of the original video. This property has two fields, which define the top-left vertex and bottom-right vertex of the rectangle. This property is optional, and the default behavior is to output the full video.
avatarConfig.videoCrop.topLeft	The top-left vertex of the rectangle for video crop. This property has two fields x and y, to define the horizontal and vertical position of the vertex. This property is required when properties.videoCrop is set.
avatarConfig.videoCrop.bottomRight	The bottom-right vertex of the rectangle for video crop. This property has two fields x and y, to define the horizontal and vertical position of the vertex. This property is required when properties.videoCrop is set.
avatarConfig.subtitleType	Type of subtitle for the avatar video file could be `external_file`, `soft_embedded`, `hard_embedded`, or `none`. This property is optional, and the default value is `soft_embedded`.
avatarConfig.backgroundImage	Add a background image using the `avatarConfig.backgroundImage` property. The value of the property should be a URL pointing to the desired image. This property is optional.
avatarConfig.backgroundColor	Background color of the avatar video, which is a string in #RRGGBBAA format. In this string: RR, GG, BB and AA mean the red, green, blue, and alpha channels, with hexadecimal value

Property	Description
	range 00~FF. Alpha channel controls the transparency, with value 00 for transparent, value FF for non-transparent, and value between 00 and FF for semi-transparent. This property is optional, and the default value is #FFFFFFFF (white).
avatarConfig.useBuiltInVoice	A boolean value indicating whether to use the voice sync for avatar as the synthesis voice. This property can only be used when using custom avatar trained with voice sync for avatar. When set to `true`, other voices specified in `synthesisConfig` or in SSML will be ignored. This property is optional, and the default value is `false`.
outputs.result	The location of the batch synthesis result file, which is a video file containing the synthesized avatar. This property is read-only.
properties.DurationInMilliseconds	The video output duration in milliseconds. This property is read-only.

Batch synthesis job properties

The following table describes the batch synthesis job properties.

Expand table

Property	Description
createdDateTime	The date and time when the batch synthesis job was created. This property is read-only.
description	The description of the batch synthesis. This property is optional.
ID	The batch synthesis job ID. This property is read-only.
lastActionDateTime	The most recent date and time when the status property value changed.

Property	Description
	This property is read-only.
properties	A defined set of optional batch synthesis configuration settings.
properties.destinationContainerUrl	The batch synthesis results can be stored in a writable Azure Blob Storage container. If you don't specify a container URI with shared access signatures (SAS) token, the Speech service stores the results in a container managed by Microsoft. SAS with stored access policies isn't supported. When the synthesis job is deleted, the result data is also deleted. This property is required when generating multiple videos in one job. For single video generation, this property is optional. This property is not included in the response when you get the synthesis job.
properties.destinationPath	The prefix path for storing batch synthesis results. If no prefix path is provided, a system-generated path will be used. This property is optional and can only be set when the `destinationContainerUrl` property is specified.
properties.timeToLiveInHours	A duration in hours after the synthesis job is created, when the synthesis results will be automatically deleted. The maximum time to live is 744 hours. The date and time of automatic deletion for synthesis jobs with a status of "Succeeded" or "Failed" is calculated as the sum of the lastActionDateTime and timeToLive properties. Otherwise, you can call the delete synthesis method to remove the job sooner.
status	The batch synthesis processing status. The status should progress from "NotStarted" to "Running", and finally to either "Succeeded" or "Failed". This property is read-only.

Text to speech properties

The following table describes the text to speech properties.

Expand table

Property	Description
customVoices	A custom voice is associated with a name and its deployment ID, like this: "customVoices": {"your-custom-voice-name": "502ac834-6537-4bc3-9fd6-140114daa66d"} You can use the voice name in your `synthesisConfig.voice` when `inputKind` is set to "PlainText", or within SSML text of inputs when `inputKind` is set to "SSML". This property is required to use a custom voice. If you try to use a custom voice that isn't defined here, the service returns an error.
inputs	The plain text or SSML to be synthesized. When the inputKind is set to `"PlainText"`, provide plain text as shown here: `"inputs": [{"content": "The rainbow has seven colors."}].` When the inputKind is set to `"SSML"`, provide text in the Speech Synthesis Markup Language (SSML) as shown here: `"inputs": [{"content": "<speak version='1.0' xml:lang='en-US'><voice xml:lang='en-US' xml:gender='Female' name='en-US-AvaMultilingualNeural'>The rainbow has seven colors.</voice></speak>"}].` Include up to 1,000 text objects if you want multiple video output files. Here's example input text that should be synthesized to two video output files: `"inputs": [{"content": "synthesize this to a file"}, {"content": "synthesize this to another file"}]`. To generate multiple videos, the output must be stored in an Azure Blob Storage container by specifying `properties.destinationContainerUrl`. You don't need separate text inputs for new paragraphs. Within any of the (up to 1,000) text inputs, you can specify new paragraphs using the "\r\n" (newline) string. Here's example input text with two paragraphs that should be synthesized to the same audio output file: `"inputs": [{"content": "synthesize this to a file\r\nsynthesize this to another paragraph in the same file"}]`. This property is required when you create a new batch synthesis job. This property isn't included in the response when you get the synthesis job.
properties.billingDetails	The number of words that were processed and billed by `customNeural` (custom voice) versus `neural` (standard voice). This property is read-only.
synthesisConfig	The configuration settings to use for batch synthesis of plain text. This property is only applicable when inputKind is set to "PlainText".

Property	Description
synthesisConfig.pitch	The pitch of the audio output. For information about the accepted values, see the adjust prosody table in the Speech Synthesis Markup Language (SSML) documentation. Invalid values are ignored. This optional property is only applicable when inputKind is set to "PlainText".
synthesisConfig.rate	The rate of the audio output. For information about the accepted values, see the adjust prosody table in the Speech Synthesis Markup Language (SSML) documentation. Invalid values are ignored. This optional property is only applicable when inputKind is set to "PlainText".
synthesisConfig.style	For some voices, you can adjust the speaking style to express different emotions like cheerfulness, empathy, and calm. You can optimize the voice for different scenarios like customer service, newscast, and voice assistant. For information about the available styles per voice, see voice styles and roles. This optional property is only applicable when inputKind is set to "PlainText".
synthesisConfig.voice	The voice that speaks the audio output. For information about the available standard voices, see language and voice support. To use a custom voice, you must specify a valid custom voice and deployment ID mapping in the customVoices property. This property is required when inputKind is set to "PlainText".
synthesisConfig.volume	The volume of the audio output. For information about the accepted values, see the adjust prosody table in the Speech Synthesis Markup Language (SSML) documentation. Invalid values are ignored. This optional property is only applicable when inputKind is set to "PlainText".
inputKind	Indicates whether the inputs text property should be plain text or SSML. The possible case-insensitive values are "PlainText" and "SSML". When the inputKind is set to "PlainText", you must also set the synthesisConfig voice property. This property is required.

Edit the background

The avatar batch synthesis API currently doesn't support setting background videos; it only supports static background images. But if you want to add a background for your video during post-production, you can generate videos with a transparent background.

To set a static background image, use the `avatarConfig.backgroundImage` property and specify a URL pointing to the desired image. Additionally, you can set the background color of the avatar video using the `avatarConfig.backgroundColor` property.

To generate a transparent background video, you must set the following properties to the required values in the batch synthesis request:

Expand table

Property	Required values for background transparency
properties.videoFormat	webm
properties.videoCodec	vp9
properties.backgroundColor	#00000000 (or `transparent`)

Clipchamp is one example of a video editing tool that supports the transparent background video generated by the batch synthesis API.

Some video editing software doesn't support the `webm` format directly and only supports `.mov` format transparent background video input, like Adobe Premiere Pro. In such cases, you first need to convert the video format from `webm` to `.mov` with a tool like FFMPEG.

FFMPEG command line:

```Bash
ffmpeg -vcodec libvpx-vp9 -i <input.webm> -vcodec png -pix_fmt rgba metadata:s:v:0
alpha_mode="1" <output.mov>
```

FFMPEG can be downloaded from ffmpeg.org. Replace `<input.webm>` and `<output.mov>` with your local path and file name in the command line.

Next steps

- Create an avatar with batch synthesis
- What is text to speech avatar

Customize text to speech avatar gestures with SSML

The Speech Synthesis Markup Language (SSML) with input text determines the structure, content, and other characteristics of the text to speech output. Most SSML tags also work in text to speech avatar. Furthermore, text to speech avatar batch mode provides avatar gesture insertion by using the SSML bookmark element with the format `<bookmark mark='gesture.*'/>`.

A gesture starts at the insertion point in time. If the gesture takes more time than the audio, the gesture is cut at the point in time when the audio is finished.

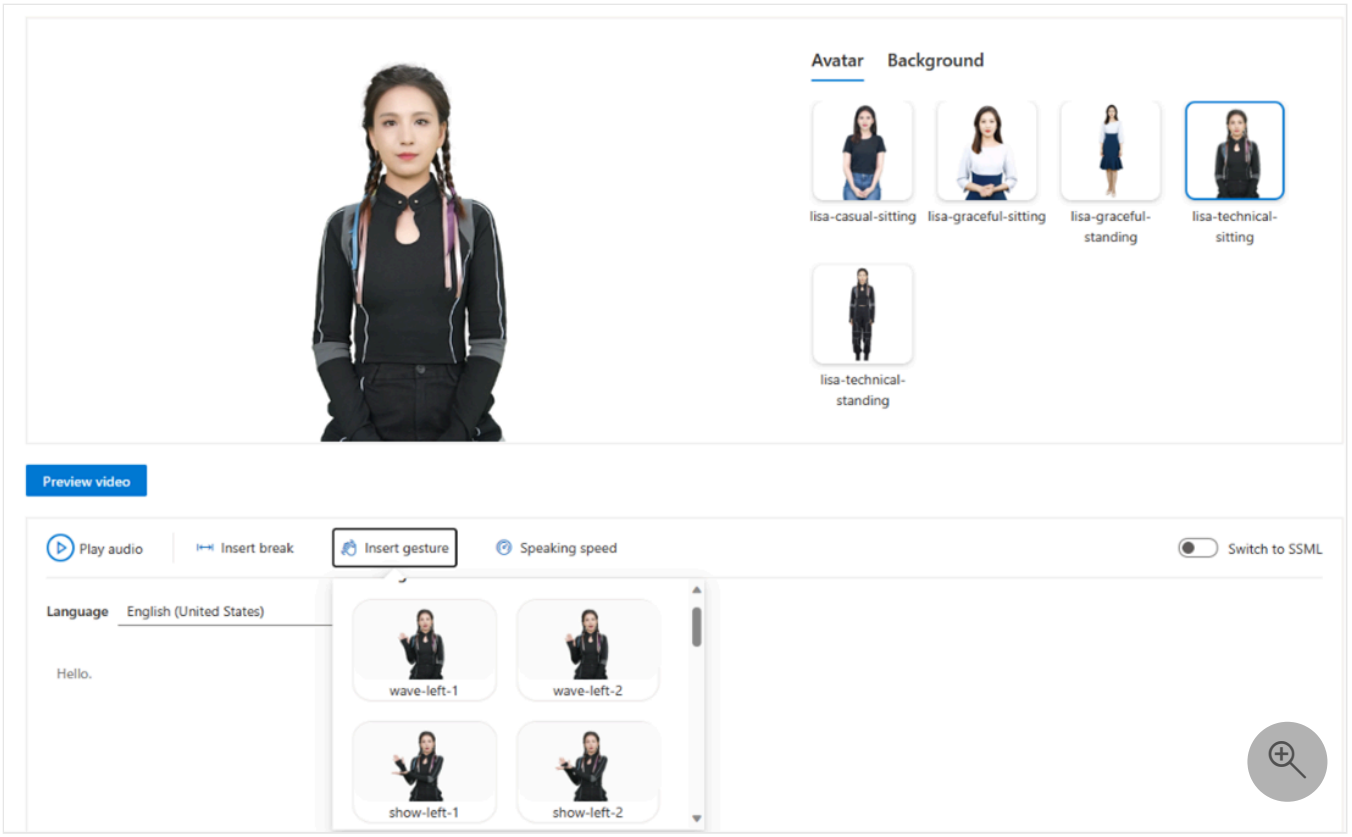
Bookmark example

The following example shows how to insert a gesture in the text to speech avatar batch synthesis with SSML.

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
<voice name="en-US-AvaMultilingualNeural">
Hello <bookmark mark='gesture.wave-left-1'/>, my name is Ava, nice to meet you!
</voice>
</speak>
```

In this example, the avatar starts waving their hand at the left after the word "Hello".



> **Note**
>
> Gesture feature isn't currently supported when a voice sync for avatar is selected in a custom text to speech avatar.

Supported standard avatar characters, styles, and gestures

The full list of standard avatar supported gestures provided here can also be found in the text to speech avatar portal.

Expand table

Characters	Styles	Gestures
Harry	business	123 calm-down come-on five-star-reviews good hello introduce invite

Characters	Styles	Gestures
		thanks welcome
Harry	casual	123 come-on five-star-reviews gong-xi-fa-cai good happy-new-year hello please welcome
Harry	youthful	123 come-on down five-star good hello invite show-right-up-down welcome
Jeff	business	123 come-on five-star-reviews hands-up here meddle please2 show silence thanks
Jeff	formal	123 come-on five-star-reviews lift please silence thanks very-good
Lisa	casual-sitting	numeric1-left-1 numeric2-left-1 numeric3-left-1 thumbsup-left-1 show-front-1 show-front-2

Characters	Styles	Gestures
		show-front-3 show-front-4 show-front-5 think-twice-1 show-front-6 show-front-7 show-front-8 show-front-9
Lisa	graceful-sitting	wave-left-1 wave-left-2 thumbsup-left show-left-1 show-left-2 show-left-3 show-left-4 show-left-5 show-right-1 show-right-2 show-right-3 show-right-4 show-right-5
Lisa	graceful-standing	
Lisa	technical-sitting	wave-left-1 wave-left-2 show-left-1 show-left-2 point-left-1 point-left-2 point-left-3 point-left-4 point-left-5 point-left-6 show-right-1 show-right-2 show-right-3 point-right-1 point-right-2 point-right-3 point-right-4 point-right-5 point-right-6
Lisa	technical-standing	
Lori	casual	123-left a-little

Characters	Styles	Gestures
		beg calm-down come-on five-star-reviews good hello open please thanks
Lori	graceful	123-left applaud come-on introduce nod please show-left show-right thanks welcome
Lori	formal	123 come-on come-on-left down five-star good hands-triangle hands-up hi hopeful thanks
Max	business	a-little-bit click-the-link display-number encourage-1 encourage-2 five-star-praise front-right good-01 good-02 introduction-to-products-1 introduction-to-products-2 introduction-to-products-3 left lower-left number-one

Characters	Styles	Gestures
		press-both-hands-down-1 press-both-hands-down-2 push-forward raise-ones-hand right say-hi shrug-ones-shoulders slide-from-left-to-right slide-to-the-left thanks the-front top-middle-and-bottom-left top-middle-and-bottom-right upper-left upper-right welcome
Max	casual	a-little-bit applaud click-the-link display-number encourage-1 encourage-2 five-star-praise front-left good-1 good-2 hello introduction-to-products-1 introduction-to-products-2 introduction-to-products-3 introduction-to-products-4 left length nodding number-one press-both-hands-down raise-ones-hand right right-front shrug-ones-shoulders slide-from-left-to-right slide-to-the-left thanks the-front upper-left

Characters	Styles	Gestures
		upper-right welcome
Max	formal	a-little-bit click-the-link display-number encourage-1 encourage-2 five-star-praise front-left front-right good-1 good-2 introduction-to-products-1 introduction-to-products-2 introduction-to-products-3 left lower-left lower-right press-both-hands-down push-forward right say-hi shrug-ones-shoulders slide-from-left-to-right slide-to-the-left the-front top-middle-and-bottom-right upper-left upper-right
Meg	formal	a-little-bit click-the-link display-number encourage-1 encourage-2 five-star-praise front-left front-right good-1 good-2 hands-forward introduction-to-products-1 introduction-to-products-2 introduction-to-products-3 left number-one press-both-hands-down-1

Characters	Styles	Gestures
		press-both-hands-down-2 right say-hi shrug-ones-shoulders slide-from-left-to-right the-front upper-left upper-right
Meg	casual	a-little-bit click-the-link cross-hand display-number encourage-1 encourage-2 five-star-praise front-left front-right good-1 good-2 handclap introduction-to-products-1 introduction-to-products-2 introduction-to-products-3 left length lower-left lower-right number-one press-both-hands-down right say-hi shrug-ones-shoulders slide-from-right-to-left slide-to-the-left spread-hands the-front top-middle-and-bottom-left top-middle-and-bottom-right upper-left upper-right
Meg	business	a-little-bit encourage-1 encourage-2 five-star-praise front-left front-right

Characters	Styles	Gestures
		good-1 good-2 introduction-to-products-1 introduction-to-products-2 introduction-to-products-3 left length number-one press-both-hands-down-1 press-both-hands-down-2 raise-ones-hand right say-hi shrug-ones-shoulders slide-from-left-to-right slide-to-the-left spread-hands thanks the-front upper-left

All styles except `lisa-graceful-sitting`, `lisa-graceful-standing`, `lisa-technical-sitting`, and `lisa-technical-standing` are supported via the real-time text to speech API. Gestures are only supported with the batch synthesis API and aren't supported via the real-time API.

Next steps

- What is text to speech avatar
- Real-time synthesis
- Use batch synthesis for text to speech avatar

Last updated on 11/09/2025

What is custom text to speech avatar?

Custom text to speech avatar allows you to create a customized, one-of-a-kind synthetic talking avatar for your application. With custom text to speech avatar, you can build a unique and natural-looking avatar for your product or brand by providing video recording data of your selected actors. The avatar is even more realistic if you also use a professional voice or voice sync for avatar for the same actor.

> **Important**
>
> Custom text to speech avatar access is **limited** based on eligibility and usage criteria. Request access on the **intake form**.

How does it work?

Creating a custom text to speech avatar requires at least 10 minutes of video recording of the avatar talent as training data, and you must first get consent from the actor talent.

The custom avatar model can support:

- Video generation via the batch synthesis API.
- Live chat via the streaming synthesis API.

Before you get started, here are some considerations:

Your use case: Do you want to use the avatar to create video content such as training material or a product introduction? Do you want to use the avatar as a virtual salesperson in a real-time conversation with your customers? There are some recording requirements for different use cases.

The look of the avatar: The custom text to speech avatar looks the same as the avatar talent in the training data, and we don't support customizing the appearance of the avatar model, such as clothes, hairstyle, etc. So if your application requires multiple styles of the same avatar, you should prepare training data for each style, as each style of an avatar is considered as a single avatar model.

The voice of the avatar: The custom text to speech avatar can work with standard voice, professional voice, and voice sync for avatar.

- Voice sync for avatar: A synthetic voice resembling the avatar talent's voice is trained alongside the custom avatar utilizing audio from the training video.

- Professional voice: Fine-tune a professional voice with more training data, providing a premium voice experience for your avatar, including natural conversations, multi-style, and multilingual support.

Here's an overview of the steps to create a custom text to speech avatar:

1. **Get consent video.** Obtain a video recording of the talent reading a consent statement. They must consent to the usage of their image and voice data to train a custom text to speech avatar model and a synthetic version of their voice.

2. **Prepare training data.** Ensure that the video recording is in the right format. It's a good idea to shoot the video recording in a professional-quality video shooting studio to get a clean background image. The quality of the resulting avatar heavily depends on the recorded video used for training. Factors like speaking rate, body posture, facial expression, hand gestures, consistency in the actor's position, and lighting of the video recording are essential to create an engaging custom text to speech avatar. See how to prepare training data for more details.

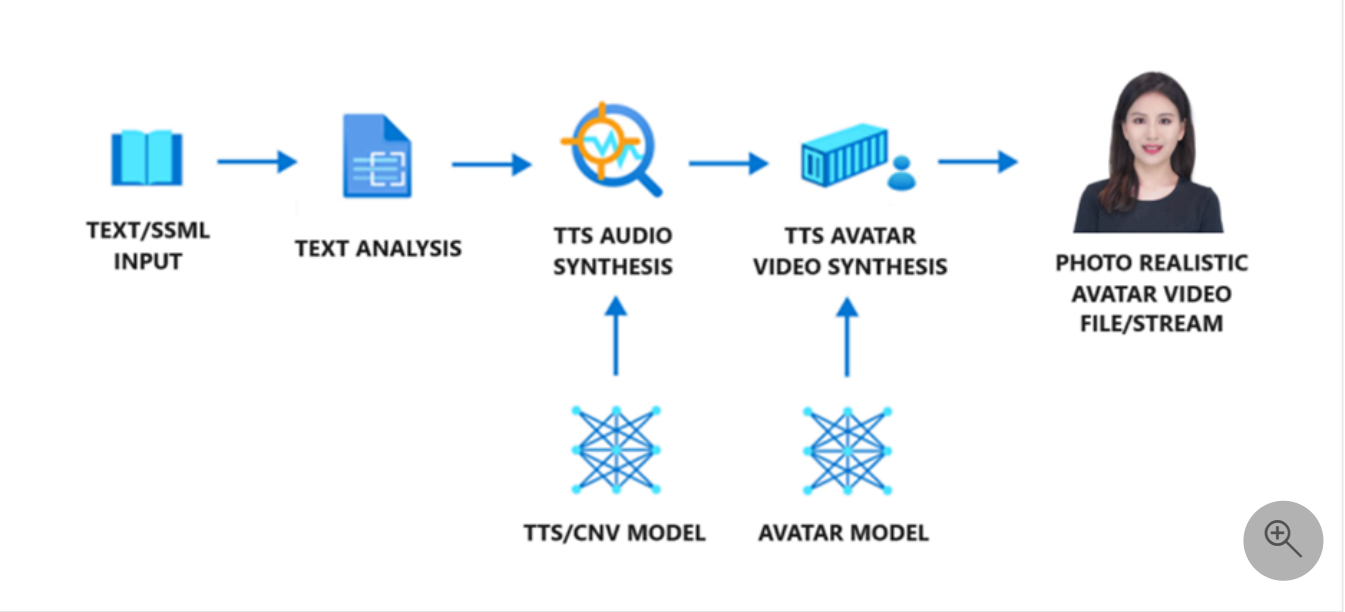
3. **Train the avatar model.** Once you have the data ready, upload your data to the custom avatar portal and start to train your model. Consent verification is conducted during the training. Make sure that you have access to the custom text to speech avatar feature before you can create a project.

4. **Deploy and use your avatar model in your applications.**

Components sequence

The custom text to speech avatar model contains three components: text analyzer, the text to speech audio synthesizer, and text to speech avatar video renderer.

- To generate an avatar video file or stream with the avatar model, text is first input into the text analyzer, which provides the output in the form of a phoneme sequence.
- The audio synthesizer synthesizes the speech audio for input text, and these two parts are provided by standard or custom voice models.
- Finally, the text to speech avatar model predicts the image of lip sync with the speech audio, so that the synthetic video is generated.



The text to speech avatar models are trained using deep neural networks based on the recording samples of human videos in different languages. All languages of standard voices and custom voices can be supported.

Available locations

For the current list of regions that support custom avatar training and usage, see the Speech service regions table.

Custom voice and custom text to speech avatar

Custom voice and custom text to speech avatar are separate features. You can use them independently or together. If you're also creating a professional voice for the actor, the avatar can be highly realistic.

The custom text to speech avatar can work with a standard voice or custom voice as the avatar's voice. For more information, see Avatar voice and language.

There are two kinds of custom voice for a custom avatar:

- **Voice sync for avatar**: When you enable the voice sync for avatar option during custom avatar training, a synthetic voice model using the likeness of the avatar talent is simultaneously trained with the avatar. This voice is exclusively associated with the custom avatar and can't be independently used. For supported regions, see the Speech service regions table.
- **Professional voice**: You can fine-tune a professional voice. Professional voice fine-tuning and custom text to speech avatar are separate features. You can use them independently or together. If you choose to use them together, you need to apply for professional voice

fine-tuning and custom text to speech avatar separately, and you're charged separately for professional voice fine-tuning and custom text to speech avatar. For more information, see the pricing page. Additionally, if you plan to use professional voice fine-tuning with a text to speech avatar, you need to deploy or copy your custom voice model to one of the avatar supported regions.

If you fine-tune a professional voice and want to use it together with the custom avatar, pay attention to the following points:

- Ensure that the custom voice endpoint is created in the same Azure AI Foundry resource as the custom avatar endpoint. As needed, refer to train your professional voice model to copy the custom voice model to the same Azure AI Foundry resource as the custom avatar endpoint.
- You can see the custom voice option in the voices list of the avatar content generation page and live chat voice settings.
- If you're using batch synthesis for avatar API, add the `"customVoices"` property to associate the deployment ID of the custom voice model with the voice name in the request. For more information, see the text to speech properties.
- If you're using real-time synthesis for avatar API, refer to our sample code on GitHub to set the custom voice.

Related content

- How to create a custom text to speech avatar
- How to prepare custom text to speech avatar training data
- Real-time synthesis for live chat avatar
- Batch synthesis for video creation
- Transparency note for text to speech

Last updated on 10/24/2025

Record video samples for custom text to speech avatar

08/07/2025

This article shows you how to prepare high-quality video samples for creating a custom text to speech avatar.

Custom text to speech avatar model building requires training on a video recording of a real human speaking. This person is the avatar talent. You must get sufficient consent under all relevant laws and regulations from the avatar talent to create a custom avatar from their talent's image or likeness. To learn about requirements of the consent statement video, see Get consent file from the avatar talent.

Recording environment

Record in a professional video recording studio or well-lit space.

Background requirements

For commercial, multi-scene avatars, use a clean, smooth, solid-colored background. A green screen works best.

If your avatar will only be used in a single scene, you can record in a specific location like your office, but you can't change the background later.

Follow these best practices when using a solid-colored background like a green screen:

- Position the green screen behind the actor. For full-body shots, place a green screen on the floor under the actor's feet. Connect the back and floor green screens seamlessly.
- Keep the green screen flat with uniform color.
- Maintain 0.5-1 meter distance between the actor and background.
- Light the green screen properly to prevent shadows.
- Keep the actor's full outline within the green screen edges.
- Don't let the actor stand too close to the green screen.
- Keep the actor's head and hands within the green screen when speaking.

Lighting requirements

- Use even, bright lighting on the actor's face. Avoid shadows on the face or reflections on glasses and clothing.

- Keep ambient lighting consistent. Turn off projectors, close curtains to avoid daylight changes, and use stable artificial light sources.

Equipment

- Camera: Minimum 1080p resolution and 25 FPS (frames per second).
- Keep lighting and camera positions fixed throughout recording.
- You can use a teleprompter during recording, but make sure it doesn't affect the actor's gaze toward the camera. Provide seating if the avatar needs to be in a sitting position.
- For half-length or seated avatars, provide seating for the actor. Choose an appropriate chair if you don't want it visible in the video.

Appearance of the actor

Custom text to speech avatar doesn't support customization of clothing or appearance. It's essential to carefully design and prepare the avatar's appearance when recording training data. Consider these tips:

Expand table

Categories	Dos	Don'ts
Hair	- The actor's hair should have a smooth and glossy surface. - Even the actor's bangs or broken hair should have a clear and smooth border. - Choose a hairstyle that is easy to keep consistent during the whole video recording.	- Avoid messy hair or backgrounds showing through the hair. - Don't let hair block the eyes or eyebrows. - Avoid shadows on the face caused by hairstyle. - Avoid hair changes too much during speech and body gesture. For example, the high ponytail of an actor might appear, disappear, and swing during speaking.
Clothing	- Pay attention to clothing status and make sure no significant changes on the clothing during speaking.	- Avoid wearing clothing and accessories that are too loose, heavy, or complex, as they might affect the consistency of clothing status during speaking and body gesture. - Avoid wearing clothing that is too similar to the background color or reflective materials like white shirts or translucent materials. - Avoid clothing with obvious lines or items with logos and brand names you don't want to highlight. - Avoid reflective elements such as metal belts, shiny leather shoes, and leather pants.
Face	- Ensure the actor's face is clearly visible.	- Avoid face obscured by hair, sunglasses, or accessories.

What video clips to record

You need these types of video clips:

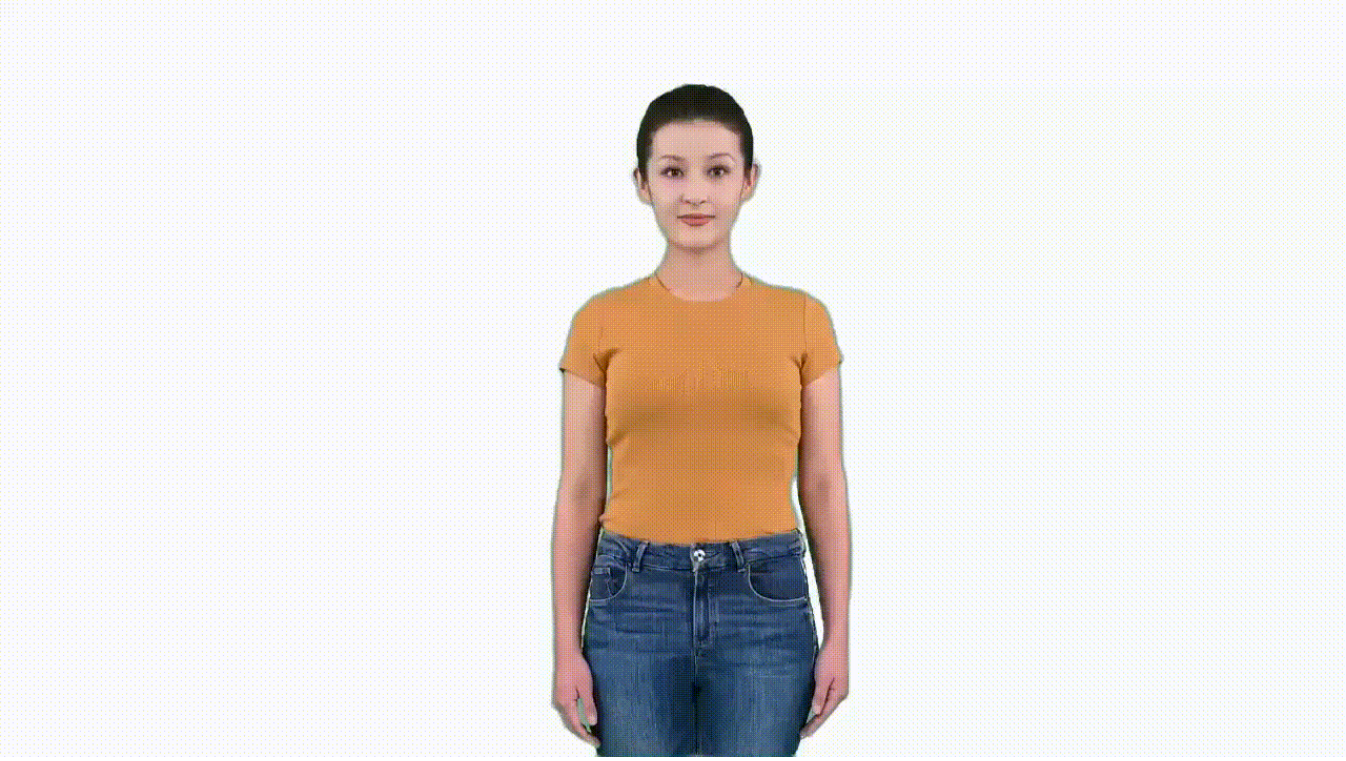
Consent Video (Required) The consent video is required for creating a custom avatar.

- The consent video must show the same avatar talent speaking and follow the consent statement requirements. Make sure the statement is recorded correctly with each word spoken clearly. You can use any supported language. To learn about consent statement video requirements, see Get consent file from the avatar talent.
- The avatar talent should always face the camera without large movements.
- Record the video in a quiet environment with clear audio at reasonable volume. Keep the signal-to-noise ratio above 20. For voice recording guidance, see the Recording custom voice samples guide.
- Make sure the actor's head isn't blocked in any frame.
- Keep other objects out of the camera view, including filming equipment and mobile phones.

Status 0 speaking (Required for gestures) The status 0 speaking video clip is required for gestures with the avatar.

- Status 0 represents the posture you can naturally maintain most of the time while speaking. For example, arms crossed in front of the body or hanging naturally at the sides.
- Maintain a front-facing pose. The actor can move slightly to show a relaxed state, like moving the head or shoulder slightly, but don't move the body too much.
- Duration: 3-5 minutes of speaking in status 0.

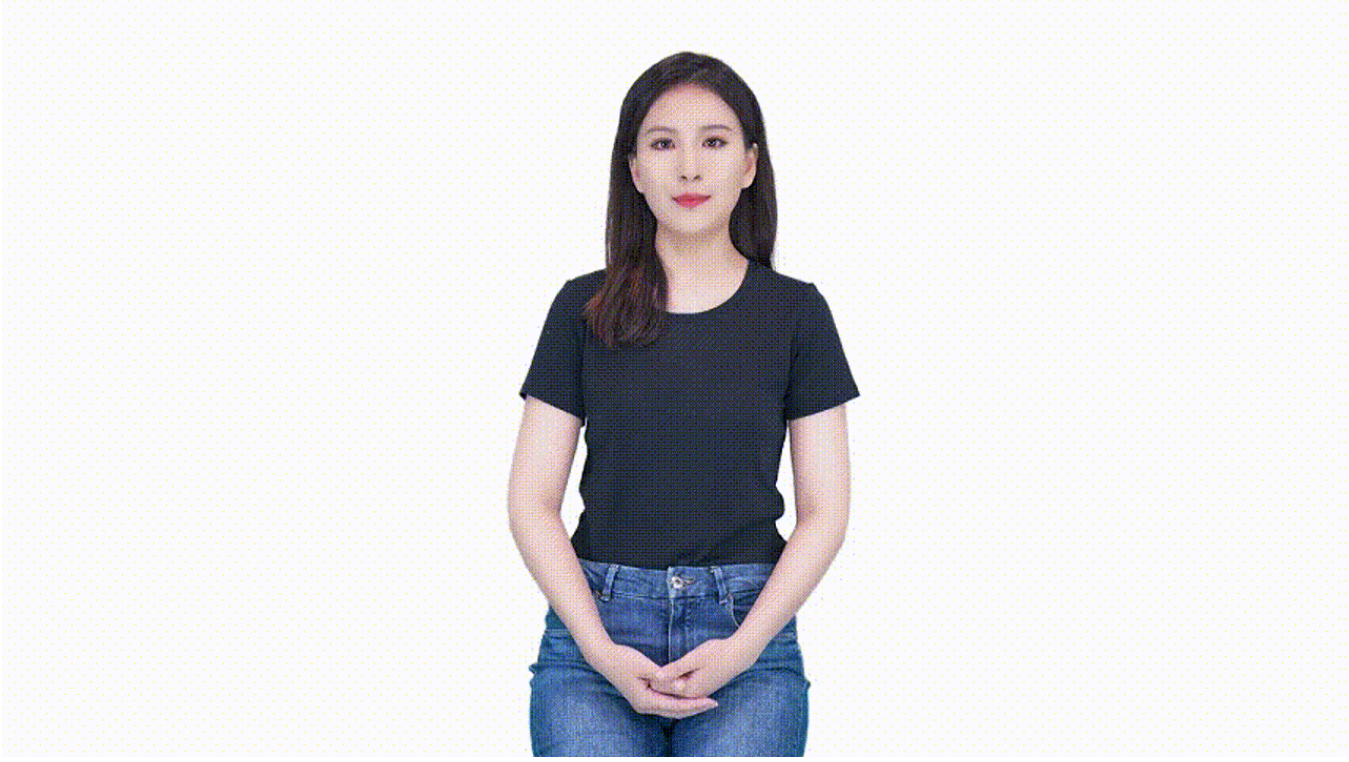
Samples of status 0 speaking



Naturally speaking (Required) The naturally speaking video clip is required for the avatar to speak naturally.

- Actor speaks in status 0 but with natural hand gestures from time to time.
- Hands should start from status 0 and return after making gestures.
- Use natural and common gestures when speaking. Avoid meaningful gestures like pointing, applause, or thumbs up.
- Duration: Minimum 5 minutes, maximum 30 minutes total. At least one 5-minute continuous video recording is required. If recording multiple clips, keep each under 10 minutes.

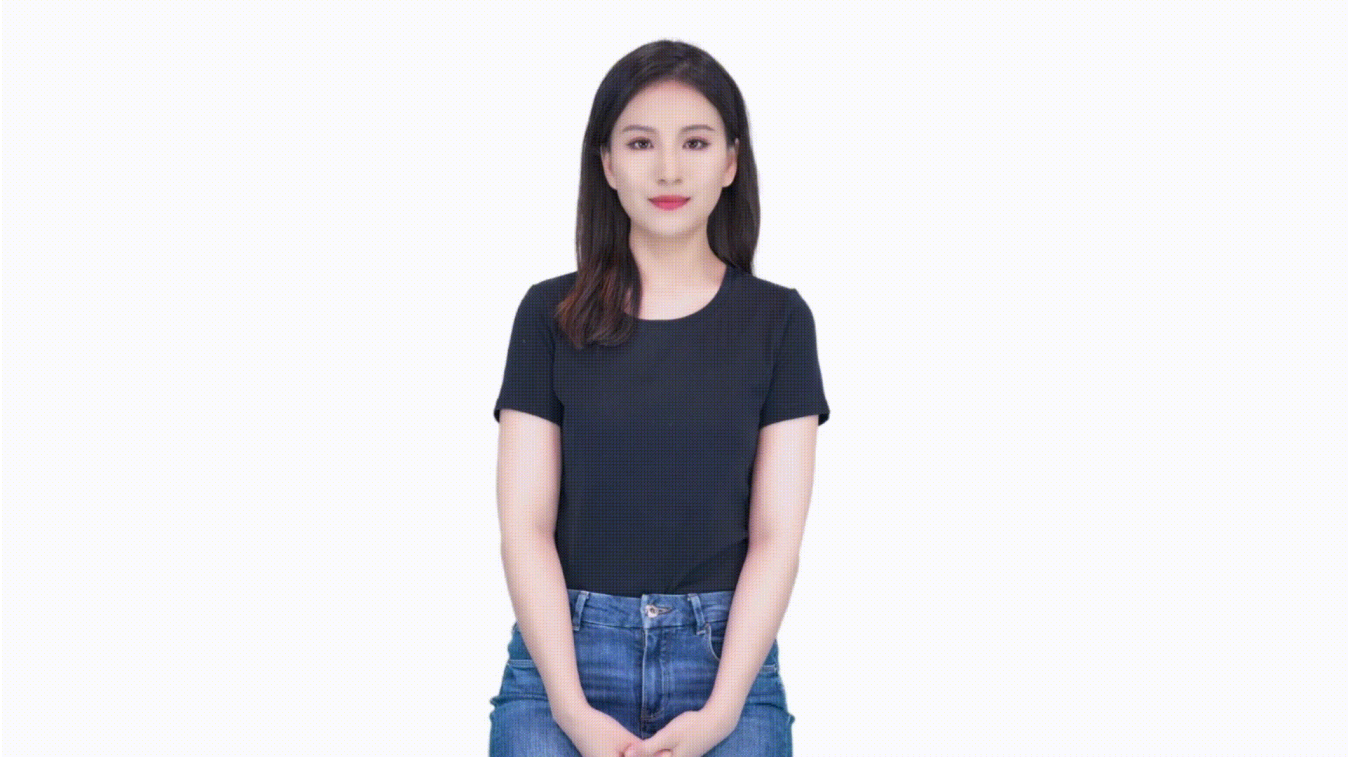
Samples of natural speaking

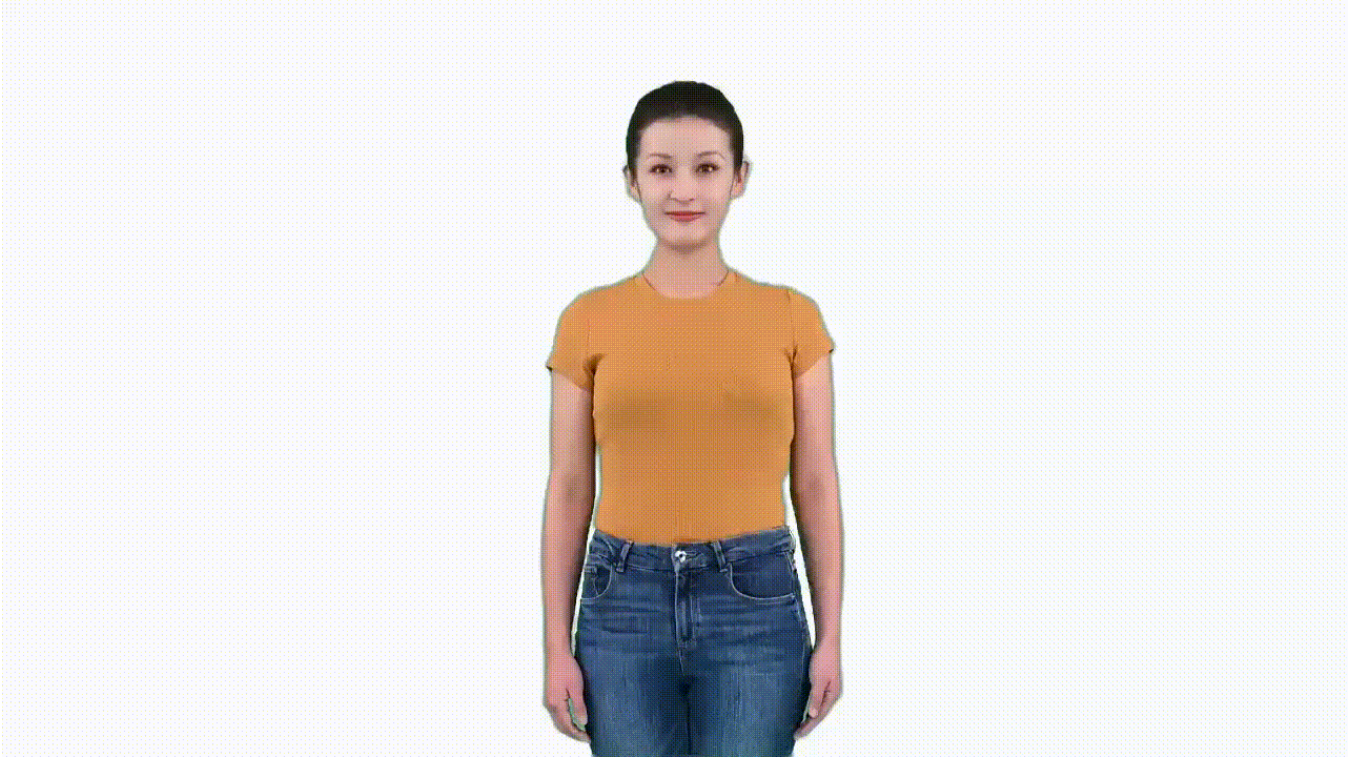


Silent status (Required) The silent status video clip is required. It's important if you build a real-time conversation with the custom avatar. The video clip is used as the main template for both speaking and listening status for a chatbot.

- Maintain status 0, don't speak, but stay relaxed.
- Even while remaining in status 0, don't stay completely still. You can move slightly but not too much. Act like you're waiting.
- Maintain a smile as if listening or waiting patiently.
- Avoid nodding frequently.
- Duration: 1 minute.

Samples of silent status





Gestures (optional)

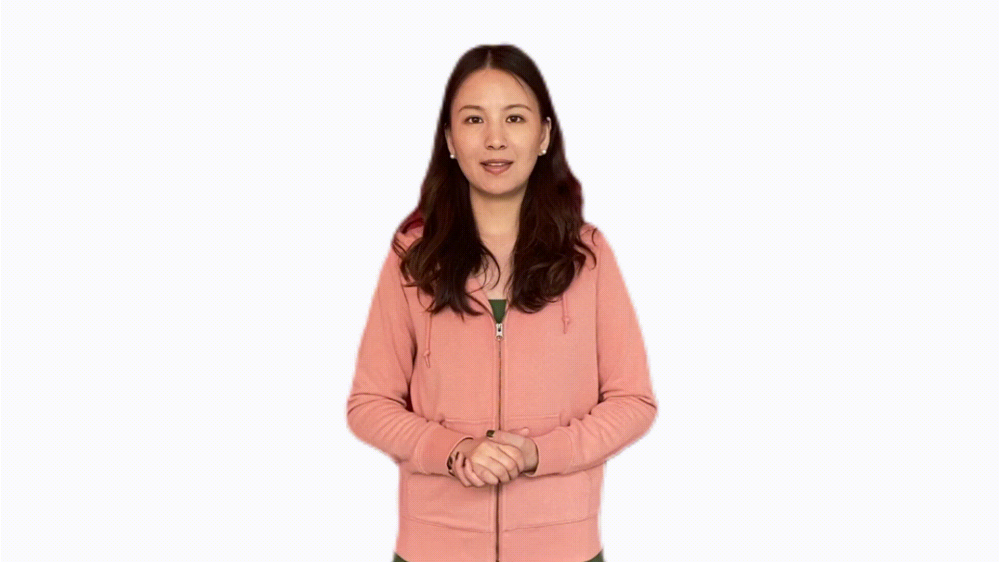
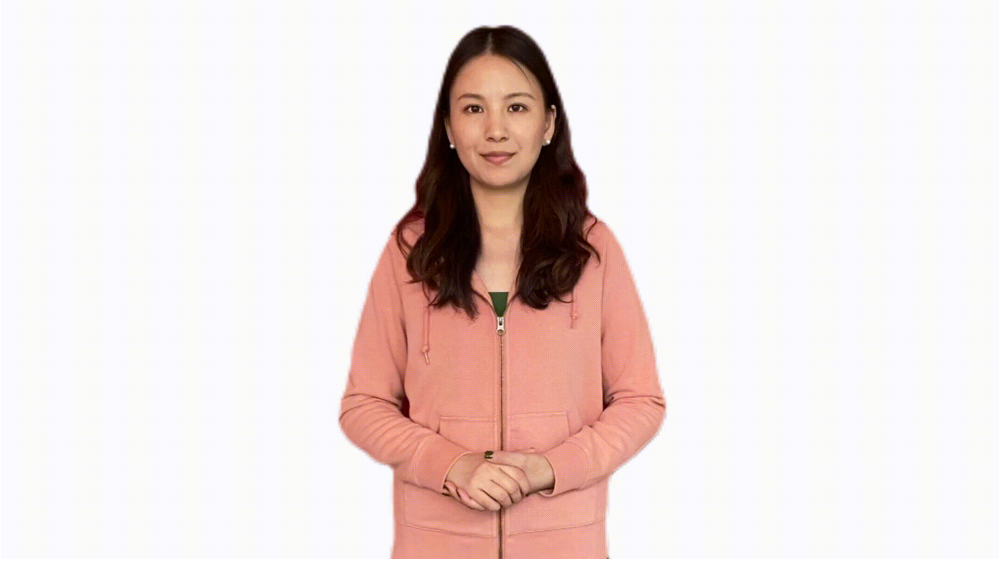
Gesture video clips are optional. If you need to insert certain gestures in the avatar speaking, follow this guideline to record gesture videos. Gesture insertion is only available for batch mode avatar; real-time avatar doesn't support gesture insertion. Each custom avatar model can support up to 10 gestures.

Gesture tips

- Each gesture clip should be within 10 seconds.
- Gestures should start from status 0 and end with status 0. It's essential that the character maintains the same position as in status 0, which is in the middle of the screen, throughout the gesture. Otherwise, the gesture clip can't be smoothly inserted into the avatar video.
- The gesture clip only captures the body gestures; the actor doesn't have to speak during making gestures.
- Design a list of gestures before recording. Here are some examples:

Samples of gesture

Expand table

Gestures	Samples
Delivering sell link/promotion code	
Praising the product	
Introducing the product	

Gestures	Samples
Displaying the price (number from 1 to 10-fist-number with each hand)	Right hand  Left hand 

High-quality avatar models are built from high-quality video recordings, including audio quality. Here are more tips for actor's performance and recording video clips:

Expand table

Dos	Don'ts
- Ensure all video clips are taken in the same conditions. - During the recording process, design the size and display area of the character you need so that the character can be displayed on the screen appropriately. - Actor should be steady during the recording. - Mind facial expressions, which should be suitable for the avatar's use case. For example, look positive and smile if the custom text to speech avatar is used as customer service. Look professionally if the avatar is used	- Don't adjust the camera parameters, focal length, position, angle of view. Don't move the camera; keep the person's position, size, angle, consistent in the camera. - Characters that are too small might lead to a loss of image quality during post-processing. Characters that are too large might cause the screen to overflow during gestures and movements.

Dos	Don'ts
for news reporting. - Maintain eye gaze towards the camera, even when using a teleprompter. - Return your body to status 0 when pausing speaking. - Speak on a self-chosen topic, and minor speech mistakes like miss a word or mispronounced are acceptable. If the actor misses a word or mispronounces something, just go back to status 0, pause for 3 seconds, and then continue speaking. - Consciously pause between sentences and paragraphs. When pausing, go back to the status 0 and close your lips. - The audio should be clear and loud enough; bad audio quality impacts training result. - Keep the shooting environment quiet.	- Don't make too long gestures or too much movement for one gesture; for example, actor's hands are always making gestures and forget to go back to status 0. - The actor's movements and gestures must not block the face. - Avoid small movements of the actor like licking lips, touching hair, talking sideways, constant head shaking during speech, and not closing up after speaking. - Avoid background noise; staff should avoid walking and talking during video recording. - Avoid other people's voice recorded during the actor speaking.

How to prepare an interaction video clip

Creating a high-quality interaction video clip is essential if you're building a real-time conversation with a custom avatar. The clip should consist of a question-and-answer format, where a photographer asks a question, and the actor responds. Loop the question-answer pair until the conversation is complete. If you're filming alone, imagine someone else asking the questions during the asking phase.

Here are some tips for each phase:

Asking phase

- Maintain status 0, don't speak, but still feel relaxed.
- Even remaining in status 0, don't keep still. Perform like you're waiting.
- Maintain a smile as if listening or waiting patiently.
- Avoid nodding frequently.
- Length: Each asking slot should last around 3–5 seconds.

Answering phase

- Speak naturally with natural hand gestures from time to time.
- Use natural and common gestures when speaking. Avoid meaningful gestures like pointing, applause, or thumbs up.
- Begin gestures after starting to speak, and stop them before you finish.
- Length: Each answering slot should last around 5 seconds.

Total video length

- Aim for a total video length of 1–5 minutes.

Data requirements

Basic video processing helps improve model training efficiency:

- Keep the character centered on screen with consistent size and position throughout recording. Keep all video processing parameters like brightness and contrast consistent. The output avatar's size, position, brightness, and contrast will directly reflect those in the training data. We don't apply alterations during processing or model building.
- Start and end clips in status 0. Actors should close their mouths, smile, and look ahead. The video should be continuous, not abrupt.

Avatar training video recording file format: .mp4 or .mov.

Resolution: At least 1920x1080.

Frame rate per second: At least 25 FPS.

Related content

- What is text to speech avatar
- What is custom text to speech avatar

How to create a custom text to speech avatar

08/07/2025

Getting started with a custom text to speech avatar is a straightforward process. All it takes are a few video clips of your actor. If you'd like to train a custom voice for the same actor, you can do so separately.

> **Note**
>
> Custom avatar access is limited based on eligibility and usage criteria. Request access on the intake form.

Prerequisites

You need an Azure AI Foundry resource in one of the regions that supports custom avatar training. Custom avatar only supports standard (S0) AI Foundry or Speech resources.

You need a video recording of the talent reading a consent statement acknowledging the use of their image and voice. You upload this video when you set up the avatar talent. For more information, see Add avatar talent consent.

You need video recordings of your avatar talent as training data. You upload these videos when you prepare training data. For more information, see Add training data.

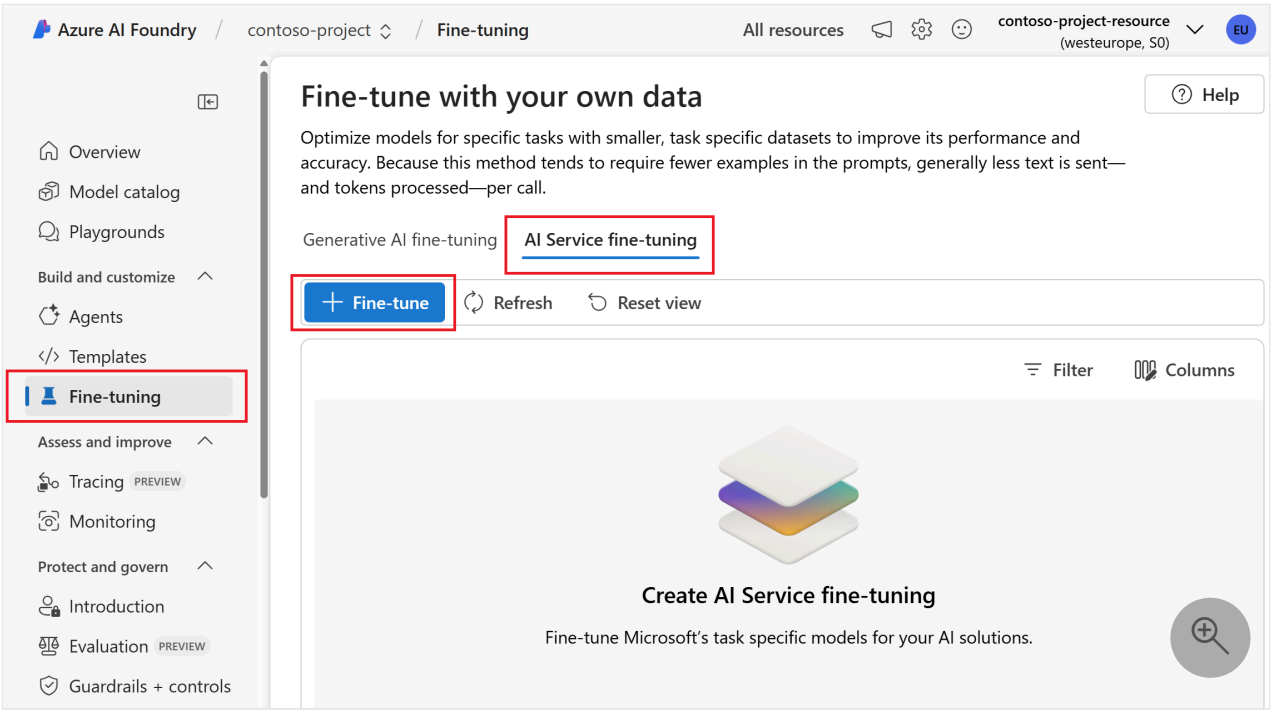
Step 1: Start fine-tuning

> **Tip**
>
> Don't mix data for different avatars in one fine-tuning workspace. Each avatar must have its own fine-tuning workspace.

To fine-tune a custom avatar, follow these steps:

1. Go to your Azure AI Foundry project in the Azure AI Foundry portal. If you need to create a project, see Create an Azure AI Foundry project.
2. Select **Fine-tuning** from the left pane.

3. Select **AI Service fine-tuning** > **+ Fine-tune**.



4. In the wizard, select **Custom avatar (text to speech avatar fine-tuning)**.

5. Select **Next**.

6. Follow the instructions provided by the wizard to create your fine-tuning workspace.

Step 2: Add avatar talent consent

An avatar talent is an individual or target actor whose video of speaking is recorded and used to create neural avatar models. You must obtain sufficient consent under all relevant laws and regulations from the avatar talent to use their video to create the custom text to speech avatar.

You must provide a video file with a recorded statement from your avatar talent, acknowledging the use of their image and voice. Microsoft verifies that the content in the recording matches the predefined script provided by Microsoft. Microsoft compares the face of the avatar talent in the recorded video statement file with randomized videos from the training datasets to ensure that the avatar talent in video recordings and the avatar talent in the statement video file are from the same person.

- If you want to create a voice sync for avatar during avatar training, a custom voice resembling your avatar is created alongside the custom avatar. The voice is used exclusively with the specified avatar. Your consent statement must include both the custom avatar and the voice sync for avatar. For an example of the consent statement for custom avatar with voice sync, see the *verbal-statement-voice-sync-for-avatar-all-locales.txt* file in the Azure-Samples/cognitive-services-speech-sdk GitHub repository.

- If you don't create a voice sync for avatar, only the custom avatar is trained, and your consent statement must reflect this scope. For an example of the consent statement for custom avatar only, see the *verbal-statement-all-locales.txt* file in the Azure-Samples/cognitive-services-speech-sdk GitHub repository.

For more information about recording the consent video, see How to record video samples and Disclosure for avatar talent.

To add an avatar talent profile and upload their consent statement in your project, follow these steps:

1. Sign in to the Azure AI Foundry portal.
2. Select **Fine-tuning** from the left pane and then select **AI Service fine-tuning**.
3. Select the custom avatar fine-tuning task (by model name) that you started as described in the previous section.
4. Select **Set up avatar talent** > **Upload consent video**.
5. On the **Upload consent video** page, follow the instructions to upload the avatar talent consent video you recorded beforehand.
    - Select the avatar type to build. Build a voice sync for avatar, which sounds like your avatar talent together with the avatar model, or build avatar without the voice sync for avatar. The option to build a voice sync for avatar is only available in the Southeast Asia, West Europe, and West US 2 regions.
    - Select the speaking language of the verbal consent statement recorded by the avatar talent.
    - Enter the avatar talent name and your company name in the same language as the recorded statement.
        - The avatar talent name must be the name of the person who recorded the consent statement.
        - The company name must match the company name that was spoken in the recorded statement.
    - You can choose to upload your data from local files, or from a shared storage with Azure Blob.
6. Select local files from your computer or enter the Azure Blob storage URL where your data is stored.
7. Select **Next**.
8. Review the upload details, and select **Upload**.

After the avatar talent consent upload is successful, you can proceed to train your custom avatar model.

Step 3: Add training data

The Speech service uses your training data to create a unique avatar tuned to match the look of the person in the recordings. After you train the avatar model, you can start synthesizing avatar videos or use it for live chats in your applications.

All data you upload must meet the requirements for the data type that you choose. To ensure that the Speech service accurately processes your data, it's important to correctly format your data before upload. To confirm that your data is correctly formatted, see Data requirements.

Upload your data

When you're ready to upload your data, go to the **Prepare training data** tab to add your data.

To upload training data, follow these steps:

1. Sign in to the Azure AI Foundry portal.
2. Select **Fine-tuning** from the left pane and then select **AI Service fine-tuning**.
3. Select the custom avatar fine-tuning task (by model name) that you started as described in the previous section.
4. Select **Prepare training data** > **Upload data**.
5. In the **Upload data** wizard, choose a data type and then select **Next**. For more information about the data types (including **Naturally Speaking**, **Silent Status**, **Gesture**, and **Status 0 speaking**), see what video clips to record.
6. Select local files from your computer or enter the Azure Blob storage URL where your data is stored.
7. Select **Next**.
8. Review the upload details, and select **Upload**.

Data files are automatically validated when you select **Upload**. Data validation includes series of checks on the video files to verify their file format, size, and total volume. If there are any errors, fix them and submit again.

After you upload the data, you can check the data overview, which indicates whether you provided enough data to start training.

Step 4: Train your avatar model

> **Important**
>
> All the training data in the project is included in the training. The model quality is highly dependent on the data you provided, and you're responsible for the video quality. Make sure you record the training videos according to the how to record video samples guide.

To create a custom avatar in the Azure AI Foundry portal, follow these steps for one of the following methods:

1. Sign in to the Azure AI Foundry portal.
2. Select **Fine-tuning** from the left pane and then select **AI Service fine-tuning**.
3. Select the custom avatar fine-tuning task (by model name) that you started as described in the previous section.
4. Select **Train model** > **+ Train model**.
5. Enter a Name to help you identify the model. Choose a name carefully. The model name is used as the avatar name in your synthesis request by the SDK and speech synthesis markup language (SSML) input. Only letters, numbers, hyphens, and underscores are allowed. Use a unique name for each model.

   > **Important**
   >
   > The avatar model name must be unique within the same Speech or AI Services resource.

6. Select **Train** to start training the model.

Training duration varies depending on how much data you use. It normally takes 20-40 compute hours on average to train a custom avatar. Check the pricing note on how training is charged.

Copy your custom avatar model to another project (optional)

Custom avatar training is currently only available in some regions. After your avatar model is trained in a supported region, you can copy it to an AI Services resource for Speech in another region as needed. For more information, see footnotes in the regions table.

> **Note**

You can only copy the voice sync for avatar model to the regions that support the voice sync for avatar feature, which are the same regions that support personal voice.

To copy your custom avatar model to another project:

1. On the **Train model** tab, select an avatar model that you want to copy, and then select **Copy to project**.
2. Select the subscription, region, AI Services resource for Speech, and project where you want to copy the model to. You must have an AI Services resource for Speech and project in the target region, otherwise you need to create them first.
3. Select **Submit** to copy the model.

Once the model is copied, you see a notification in the Azure AI Foundry portal.

Navigate to the project where you copied the model to deploy the model copy.

Step 5: Deploy and use your avatar model

After you successfully created and trained your avatar model, you deploy it to your endpoint.

To deploy your avatar:

1. Sign in to the Azure AI Foundry portal.
2. Select **Fine-tuning** from the left pane and then select **AI Service fine-tuning**.
3. Select the custom avatar fine-tuning task (by model name) that you started as described in the previous section.
4. Select **Deploy model** > **Deploy model**.
5. Select a model that you want to deploy.
6. Select **Deploy** to start the deployment.

> **Important**
>
> When a model is deployed, you pay for continuous up time of the endpoint regardless of your interaction with that endpoint. Check the pricing note on how model deployment is charged. You can delete a deployment when the model isn't in use to reduce spending and conserve resources.

After you deploy your custom avatar, it's available to use in the Azure AI Foundry portal or via API:

- The avatar appears in the avatar list of text to speech avatar on Azure AI Foundry portal.
- The avatar appears in the avatar list of live chat avatars via Azure AI Foundry portal.
- You can call the avatar from the SDK and SSML input by specifying the avatar model name. For more information, see the avatar properties.

Remove a deployment

To remove your deployment, follow these steps:

1. Sign in to the Azure AI Foundry portal.
2. Select **Fine-tuning** from the left pane and then select **AI Service fine-tuning**.
3. Select the custom avatar fine-tuning task (by model name) that you started as described in the previous section.
4. Select the deployment on the **Deploy model** page. The model is actively hosted if the status is "Succeeded".
5. You can select the **Delete deployment** button and confirm the deletion to remove the hosting.

> **Tip**
>
> Once a deployment is removed, you no longer pay for its hosting. Deleting a deployment doesn't cause any deletion of your model. If you want to use the model again, create a new deployment.

Next steps

- What is text to speech avatar
- How to record video samples

Text to speech with the audio content creation tool

08/05/2025

You can use the audio content creation tool in Azure AI Foundry portal or Speech Studio for text to speech without writing any code.

> **Tip**
>
> Select **Foundry portal** or **Speech Studio** at the top of this article.

Build highly natural audio content for various scenarios, such as audiobooks, news broadcasts, video narrations, and chat bots. With audio content creation, you can efficiently fine-tune text to speech voices and design customized audio experiences.

The tool is based on Speech Synthesis Markup Language (SSML). It allows you to adjust text to speech output attributes in real-time or batch synthesis, such as voice characters, voice styles, speaking speed, pronunciation, and prosody.

- No-code approach: You can use the audio content creation tool for text to speech synthesis without writing any code. The output audio might be the final deliverable that you want. For example, you can use the output audio for a podcast or a video narration.
- Developer-friendly: You can listen to the output audio and adjust the SSML to improve speech synthesis. Then you can use the Speech SDK or Speech CLI to integrate the SSML into your applications.

You have easy access to a broad portfolio of languages and voices. These voices include state-of-the-art standard voices and your custom voice, if you built one.

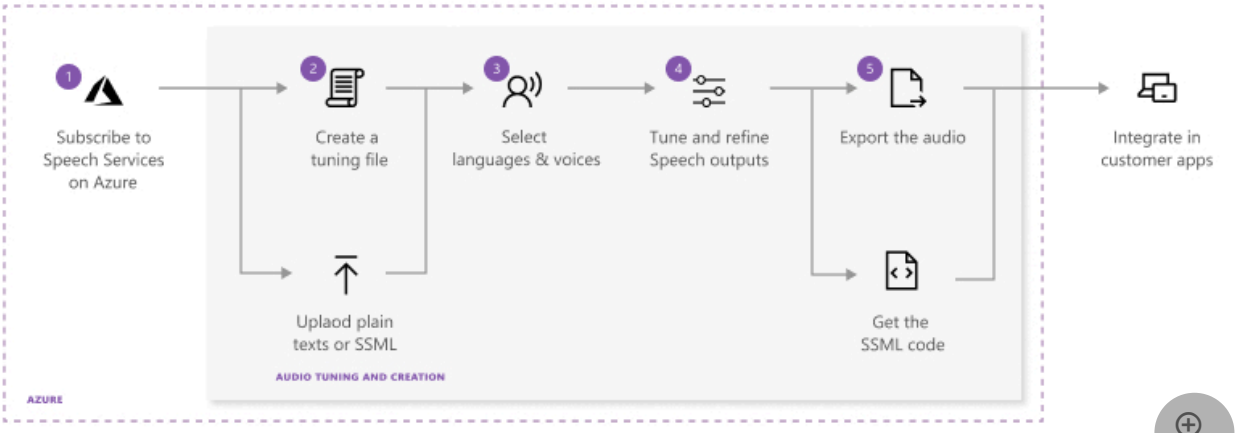
The audio content creation tool is free to access; you pay only for Speech service usage.

Prerequisites

- An active Azure subscription. Create one for free.
- Permission to create resources in your subscription.
- An Azure AI Foundry project. For more information, see Create an Azure AI Foundry project.

Use the audio content creation tool

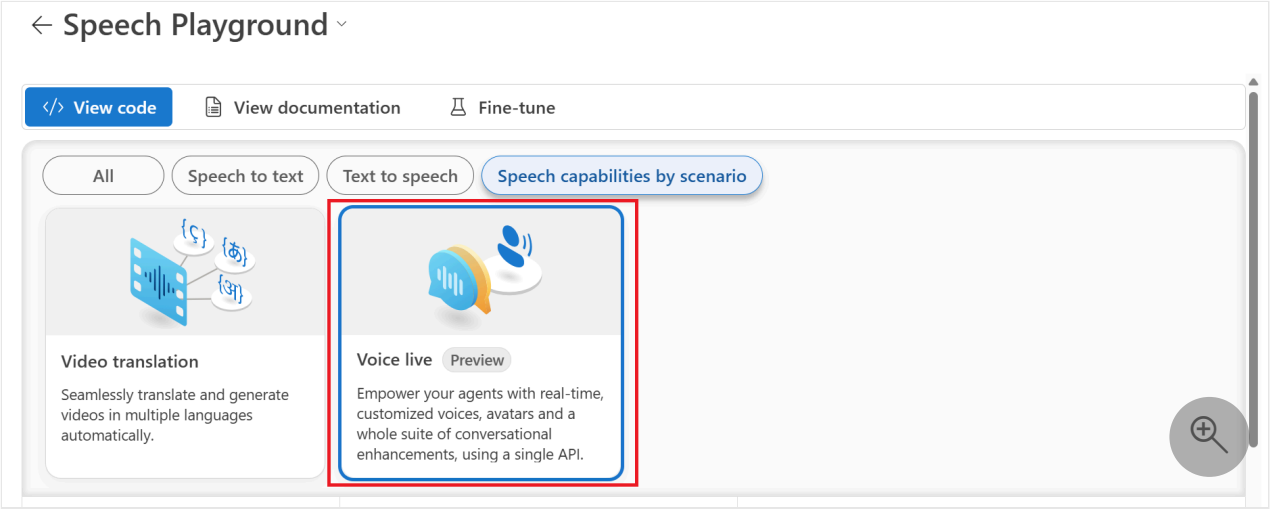
The following diagram displays the process for fine-tuning the text to speech outputs.



Access the tool

To access the audio content creation tool in Azure AI Foundry, follow these steps:

1. Go to your project in Azure AI Foundry.
2. Select **Playgrounds** from the left pane.
3. In the **Speech playground** tile, select **Try the Speech playground**.
4. Select **Text to speech** > **Audio content creation**. You might need to scroll to find the tile.



Workflow overview

Once you have access to the tool, follow this general workflow:

1. Create an audio tuning file by using plain text or SSML scripts. Enter or upload your content into audio content creation.

2. Choose the voice and the language for your script content. Audio content creation includes all of the standard text to speech voices. You can use standard voices or a custom voice.

   > **Note**
   >
   > Custom voice access is **limited** based on eligibility and usage criteria. Request access on the **intake form**.

3. Select the content you want to preview, and then select **Play** (via the triangle icon) to preview the default synthesis output.

   If you make any changes to the text, select the **Stop** icon, and then select **Play** again to regenerate the audio with changed scripts.

   Improve the output by adjusting pronunciation, break, pitch, rate, intonation, voice style, and more. For a complete list of options, see Speech Synthesis Markup Language.

4. Save and export your tuned audio.

   When you save the tuning track in the system, you can continue to work and iterate on the output. When you're satisfied with the output, you can create an audio creation task with the export feature. You can observe the status of the export task and download the output for use with your apps and products.

Create an audio tuning file

You can get your content into the audio content creation tool in either of two ways:

Option 1: Create a new audio tuning file

1. Select **New** > **Text file** to create a new audio tuning file.

2. Enter or paste your content into the editing window. The allowable number of characters for each file is 20,000 or fewer. If your script contains more than 20,000 characters, you can use Option 2 to automatically split your content into multiple files.

3. Select **Save**.

Option 2: Upload an audio tuning file

1. Select **Upload** > **Text file** to import one or more text files. Both plain text and SSML are supported.

   If your script file is more than 20,000 characters, split the content by paragraphs, by characters, or by regular expressions.

2. When you upload your text files, make sure that they meet these requirements:

Expand table

Property	Description
File format	Plain text (.txt) or SSML text (.txt) Zip files aren't supported.
Encoding format	UTF-8
File name	Each file must have a unique name. Duplicate files aren't supported.
Text length	Character limit is 20,000. If your files exceed the limit, split them according to the instructions in the tool.
SSML restrictions	Each SSML file can contain only a single piece of SSML.

Here's a plain text example:

```txt
Welcome to use audio content creation to customize audio output for your products.
```

Here's an SSML example:

```xml
<speak xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="http://www.w3.org/2001/mstts" version="1.0" xml:lang="en-US">
   <voice name="en-US-AvaMultilingualNeural">
   Welcome to use audio content creation <break time="10ms" />to customize audio output for your products.
   </voice>
</speak>
```

Export tuned audio

After you review your audio output and are satisfied with your tuning and adjustment, you can export the audio.

1. Select **Export** to create an audio creation task.

   We recommend **Export to Audio library** to easily store, find, and search audio output in the cloud. You can better integrate with your applications through Azure blob storage. You can also download the audio to your local disk directly.

2. Choose the output format for your tuned audio. The **supported audio formats and sample rates** are listed in the following table:

   Expand table

Format	8 kHz sample rate	16 kHz sample rate	24 kHz sample rate	48 kHz sample rate
wav	riff-8khz-16bit-mono-pcm	riff-16khz-16bit-mono-pcm	riff-24khz-16bit-mono-pcm	riff-48khz-16bit-mono-pcm
mp3	N/A	audio-16khz-128kbitrate-mono-mp3	audio-24khz-160kbitrate-mono-mp3	audio-48khz-192kbitrate-mono-mp3

3. To view the status of the task, select the **Task list** tab.

   If the task fails, see the detailed information page for a full report.

4. When the task is complete, your audio is available for download on the **Audio library** pane.

5. Select the file you want to download and **Download**.

Now you're ready to use your custom tuned audio in your apps or products.

Related content

- Speech Synthesis Markup Language (SSML)
- Batch synthesis

What are OpenAI text to speech voices?

Like Azure AI Speech voices, OpenAI text to speech voices deliver high-quality speech synthesis to convert written text into natural sounding spoken audio. This unlocks a wide range of possibilities for immersive and interactive user experiences.

OpenAI text to speech voices are available via two model variants: `Neural` and `NeuralHD`.

- `Neural`: Optimized for real-time use cases with the lowest latency, but lower quality than `NeuralHD`.
- `NeuralHD`: Optimized for quality.

Available text to speech voices in Azure AI services

You might ask: If I want to use an OpenAI text to speech voice, should I use it via the Azure OpenAI in Azure AI Foundry Models or via Azure AI Speech? What are the scenarios that guide me to use one or the other?

Each voice model offers distinct features and capabilities, allowing you to choose the one that best suits your specific needs. You want to understand the options and differences between available text to speech voices in Azure AI services.

You can choose from the following text to speech voices in Azure AI services:

- OpenAI text to speech voices in Azure OpenAI. For the current list of supported regions, see the Speech service regions table.
- OpenAI text to speech voices in Azure AI Speech. For the current list of supported regions, see the Speech service regions table.
- Azure AI Speech service text to speech voices. Available in dozens of regions. See the region list.

OpenAI text to speech voices via Azure OpenAI or via Azure AI Speech?

If you want to use OpenAI text to speech voices, you can choose whether to use them via Azure OpenAI or via Azure AI Speech. You can visit the Voice Gallery to listen to samples of Azure OpenAI voices or synthesize speech with your own text using the Audio Content Creation. The audio output is identical in both cases, with only a few feature differences between the two services. See the table below for details.

Here's a comparison of features between OpenAI text to speech voices in Azure OpenAI and OpenAI text to speech voices in Azure AI Speech.

Expand table

Feature	Azure OpenAI (OpenAI voices)	Azure AI Speech (OpenAI voices)	Azure AI Speech voices
Region	North Central US, Sweden Central	North Central US, Sweden Central	Available in dozens of regions. See the region list.
Voice variety	6	12	More than 500
Multilingual voice number	6	12	49
Max multilingual language coverage	57	57	77
Speech Synthesis Markup Language (SSML) support	Not supported	Support for a subset of SSML elements.	Support for the full set of SSML in Azure AI Speech.
Development options	REST API	Speech SDK, Speech CLI, REST API	Speech SDK, Speech CLI, REST API
Deployment option	Cloud only	Cloud only	Cloud, embedded, hybrid, and containers.
Real-time or batch synthesis	Real-time	Real-time	Real-time and batch synthesis
Latency	greater than 500 ms	greater than 500 ms	less than 300 ms
Sample rate of synthesized audio	24 kHz	8, 16, 24, and 48 kHz	8, 16, 24, and 48 kHz
Speech output audio format	opus, mp3, aac, flac	opus, mp3, pcm, truesilk	opus, mp3, pcm, truesilk

There are additional features and capabilities available in Azure AI Speech that aren't available with OpenAI voices. For example:

- OpenAI text to speech voices in Azure AI Speech only support a subset of SSML elements. Azure AI Speech voices support the full set of SSML elements.
- Azure AI Speech supports word boundary events. OpenAI voices don't support word boundary events.

Available OpenAI text to speech voices

The available OpenAI voices in Azure OpenAI are:

- `alloy`
- `echo`
- `fable`
- `onyx`
- `nova`
- `shimmer`

The available OpenAI voices in Azure AI Speech are:

- `en-US-AlloyMultilingualNeural`
- `en-US-EchoMultilingualNeural`
- `en-US-FableMultilingualNeural`
- `en-US-OnyxMultilingualNeural`
- `en-US-NovaMultilingualNeural`
- `en-US-ShimmerMultilingualNeural`
- `en-US-AlloyMultilingualNeuralHD`
- `en-US-EchoMultilingualNeuralHD`
- `en-US-FableMultilingualNeuralHD`
- `en-US-OnyxMultilingualNeuralHD`
- `en-US-NovaMultilingualNeuralHD`
- `en-US-ShimmerMultilingualNeuralHD`

SSML elements supported by OpenAI text to speech voices in Azure AI Speech

The Speech Synthesis Markup Language (SSML) with input text determines the structure, content, and other characteristics of the text to speech output. For example, you can use SSML to define a paragraph, a sentence, a break or a pause, or silence. You can wrap text with event tags such as bookmark or viseme that can be processed later by your application.

The following table outlines the Speech Synthesis Markup Language (SSML) elements supported by OpenAI text to speech voices in Azure AI speech. Only the following subset of SSML tags are supported for OpenAI voices. See SSML document structure and events for more information.

Expand table

SSML element name	Description
`<speak>`	Encloses the entire content to be spoken. It's the root element of an SSML document.
`<voice>`	Specifies a voice used for text to speech output.
`<sub>`	Indicates that the alias attribute's text value should be pronounced instead of the element's enclosed text.
`<say-as>`	Indicates the content type, such as number or date, of the element's text. All of the `interpret-as` property values are supported for this element except `interpret-as="name"`. For example, `<say-as interpret-as="date" format="dmy">10-12-2016</say-as>` is supported, but `<say-as interpret-as="name">ED</say-as>` isn't supported. For more information, see pronunciation with SSML.
`<s>`	Denotes sentences.
`<lang>`	Indicates the default locale for the language that you want the neural voice to speak.
`<break>`	Use to override the default behavior of breaks or pauses between words.

Related content

- Try the text to speech quickstart in Azure AI Speech
- Try the text to speech via Azure OpenAI

Last updated on 10/24/2025

Text to speech FAQ

This article answers commonly asked questions about the text to speech (TTS) capability. If you can't find answers to your questions here, check out other support options.

General

How does the billing work for text to speech?

Text to speech usage is billed per character. Check the definition of billable characters in the pricing note.

What is the rate limit for the text to speech synthesis requests?

The text to speech synthesis rate scales automatically as it receives more requests. A default rate limit is set per speech resource. The rate is adjustable with business justifications and no extra charges are incurred for rate limit increase. Check more details in Speech service quotas and limits.

How would we disclose to the end user that the voice is a synthetic voice?

We recommend that every user should follow our code of conduct when using the text to speech capability. There are several ways to disclose the synthetic nature of the voice including implicit and explicit byline. Refer to Disclosure design guidelines.

How can I reduce the latency for my voice app?

We provide several tips for you to lower the latency and bring the best performance to your users. See Lower speech synthesis latency using Speech SDK.

What output audio formats does text to speech support?

Azure AI text to speech supports various streaming and non-streaming audio formats, with the commonly used sampling rates. All TTS standard voices are created to support high-fidelity audio outputs with 48 kHz and 24 kHz. The audio can be resampled to support other rates as needed. See Audio outputs.

Can the voice be customized to stress specific words?

Adjusting the emphasis is supported for some voices depending on the locale. See the emphasis tag.

Can we have multiple strength for each emotion, like sad, slightly sad, and so on, in?

Adjusting the style degree is supported for some voices depending on the locale. See the mstts:express-as tag.

Is there a mapping between Viseme IDs and mouth shape?

Yes. See Get facial position with viseme.

Why am I getting HTTP 429 (Too Many Requests) errors when using Text-to-Speech Standard Voice?

The default quota of 200 transactions per second (TPS) for Text-to-Speech (TTS) Standard Voice is designed to accommodate the needs of most customers. In most cases, HTTP 429 errors are not caused by quota limitations, but by insufficient backend service capacity in the selected region for the specified voice. Increasing the quota does not resolve these capacity constraints. To address this issue effectively:

- Use native regions: Deploy the voice in a region where it is natively supported and better resourced (for example, use the Japan region for Japanese voices).
- Select popular voices: Choose a more commonly used voice within your current region to reduce the likelihood of hitting capacity limits.

Audio Content Creation

How can I reference a lexicon file that I created on the Audio Content Creation platform in my code?

First, you can open the lexicon file on the Audio Content Creation and obtain the lexicon file ID, which is located before "?fileKind=CustomLexiconFile" in the file path. For example, if the file

path is `https://speech.microsoft.com/portal/d391a094f76846acbcd11dc2ba835f4f/audiocontentcreation/file/6cbc2527-8d57-4c1b-b9d9-3ea6d13ca95c?fileKind=CustomLexiconFile`, the lexicon file ID is `6cbc2527-8d57-4c1b-b9d9-3ea6d13ca95c`. Then, switch a file referencing this lexicon to SSML format on the Audio Content Creation. In the SSML file, locate the `<!--ID=FCB` xml node, where you can find the URI of the lexicon file based on the mentioned file ID. Finally, reference the lexicon file URI link using the SSML lexicon element in your code. For instance, if you locate the XML node `<!--ID=FCB5B6FB566-33CA-4B68-BEAF-B013C53B3368;Version=1|{"Files":{"6cbc2527-8d57-4c1b-b9d9-3ea6d13ca95c":{"FileKind":"CustomLexiconFile","FileSubKind":"CustomLexiconFile","Uri":"https://cvoiceprodwus2.blob.core.windows.net/acc-public-files/d391a094f76846acbcd11dc2ba835f4f/e9a6a5a2-9cef-47f4-b961-d175be75d92f.xml"}}}`, you can obtain the lexicon file URI `https://cvoiceprodwus2.blob.core.windows.net/acc-public-files/d391a094f76846acbcd11dc2ba835f4f/e9a6a5a2-9cef-47f4-b961-d175be75d92f.xml`.

Professional voice fine-tuning

How much data is required for professional voice fine-tuning?

You need training data of at least 300 lines of recordings (or approximately 30 minutes of speech) for professional voice fine-tuning. We recommend 2,000 lines of recordings (or approximately 2-3 hours of speech) to create a voice for production use. For the script selection criteria, see Record custom voice samples.

Can we include duplicate text sentences in the same set of training data?

No. The service will flag the duplicate sentences and just keep the first imported one. For the script selection criteria, see Record custom voice samples.

Can we include multiple styles in the same set of training data?

We recommend that you keep the style consistent in one set of training data. If the styles are different, put them into different training sets. In this case, consider using the multi-style training method of professional voice fine-tuning. For the script selection criteria, see Record custom voice samples.

Does switching styles via SSML work for custom voices?

Switching styles via SSML works for both multi-style standard voices and multi-style custom voices. With multi-style training, you can create a voice that speaks in different styles, and you can also adjust these styles via SSML.

How does cross-lingual voice work with languages that have different pronunciation structure and assembly?

Sentence structure and pronunciation naturally vary across languages such as English and Japanese. Each neural voice is trained with audio data recorded by native speaking voice talent. For cross lingual voice, we transfer the major features like timbre to sound like the original speaker and preserve the right pronunciation. For example, a cross-lingual voice uses the native way to speak Japanese and still sounds similar (but not exactly) like the original English speaker.

Can I use professional voice fine-tuning to customize pronunciation for my domain?

Professional voice fine-tuning enables you to create a brand voice for your business. You can optimize it for your domain as well. We recommend you include domain-specific samples in your training data for higher naturalness. However, the pronunciation is defined by the Speech service by default. We don't support pronunciation customization with professional voice fine-tuning. If you want to customize pronunciation for your voice, use SSML. See Pronunciation with Speech Synthesis Markup Language (SSML).

After one training can I train my voice again?

You can train again. Each training creates a new voice model. You are charged for each training.

Is the model version the same as the engine version?

No. The model version is different from the engine version. The model version means the version of the training recipe for your model and varies by the features supported and model training time. Azure AI services text to speech engines are updated from time to time to capture the latest language model that defines the pronunciation of the language. After you've trained your voice, you can apply your voice to the new language model by updating to the